Big Data Streaming Project

Ralf Cedric T. Zabala
John Bernard B. Barce
AM3

Tabel of Contents

Part 0 - Initial Installation and Setup for Kafka, Spark, and Cassandra **4**

Part 0.1 - Setting up the JDK for Kafka and Spark 4

Update Ubuntu 4

Install JDK 4

Show Java version 5

Part 0.2 - Installing and Setting Up Kafka 6

Going to Apache Kafka 6

Extract Kafka 7

Starting ZooKeeper server 7

Starting the Kafka Server 8

Creating a topic, producer, and consumer 9

Test 11

Part 0.3 - Installing and Setting Up Spark 12

Installing and Extract Spark 12

Setup Spark 12

Create a Dedicated Spark User 13

Create systemd Service Files 13

Start Spark to test 15

Spark-Kafka Integration 16

Start Spark and Import the needed libraries 18

Part 0.3: Install and Configure Apache Cassandra 19

APT Method 19

Install Cassandra using wget and extract the tar.gz 19

Extract the tar 19

Create the Startup Script 20

Run script 20

Part 1 **22**

Start zookeeper and kafka 22

Create Topic 22

Producer and Consumer 23

Create 10 instances of the Producer 25

How the Script works 26

Run created script for 10 copies 27

Run all 10 Producers 30

Create Keyspace 32

Create Table 33

Part 2 **35**

Code of fetch_stream_to_cassandra.py 35

Run Spark 37

Checking the Spark UI 38

Check Cassandra Table 38

Run 10 producers 39

Updated Spark UI and Console 40

Going back to Cassandra 40

References: **43**

Part 0 - Initial Installation and Setup for Kafka, Spark, and Cassandra

Before proceeding with the main Big Data Streaming Pipeline, it is essential to set up and install the necessary tools, including Apache Kafka, Spark, and Cassandra. That is why Part 0 will provide an initial installation guide for the aforementioned tools, which are necessary before proceeding with the main project. **If you already have these installed**, you can **skip this** part and proceed to the main steps for Big Data Streaming.

Part 0.1 - Setting up the JDK for Kafka and Spark

This is an important step since Kafka and Spark run on a Java Virtual Machine (JVM), so installing a Java Development Kit is necessary for them to function properly, and without it, their services cannot operate (DataCamp, 2024). The selected Java version will be **JDK 17** since both versions of Kafka and Spark can run on it; however, this setup also uses Cassandra. **JDK 11** will also be installed, but will not be used as the default JDK for this system, and instead will be integrated in a bash script for setting up Cassandra, which will be further elaborated in the Cassandra installation section.

Update Ubuntu

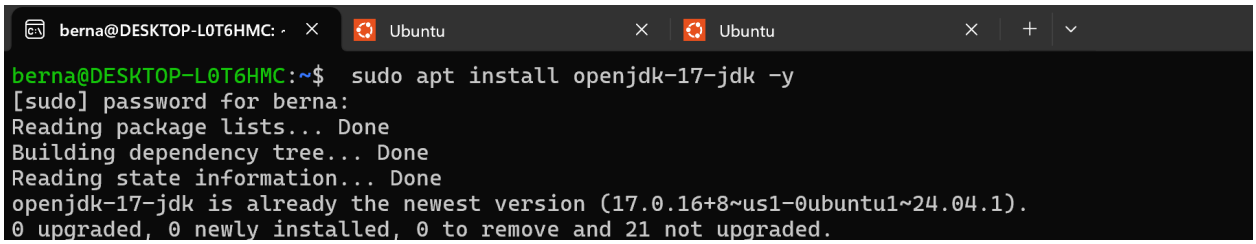
Before proceeding with any installation in Ubuntu, it is important to update Ubuntu itself by inputting the command `sudo apt update`, in which Ubuntu will then update itself in order to have the latest packages and metadata installed.

```
berna@DESKTOP-L0T6HMC:~$ sudo apt update
[sudo] password for berna:
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1260 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1541 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [208 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.5 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [8984 B]
Get:10 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [904 kB]
Get:11 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [202 kB]
Get:12 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [52.3 kB]
Get:13 http://security.ubuntu.com/ubuntu noble-security/universe amd64 c-n-f Metadata [19.3 kB]
Get:14 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [2069 kB]
```

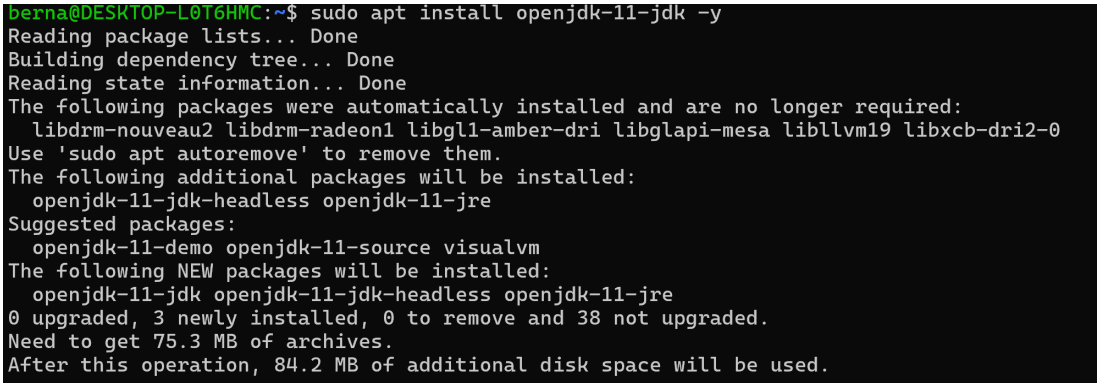
Install JDK

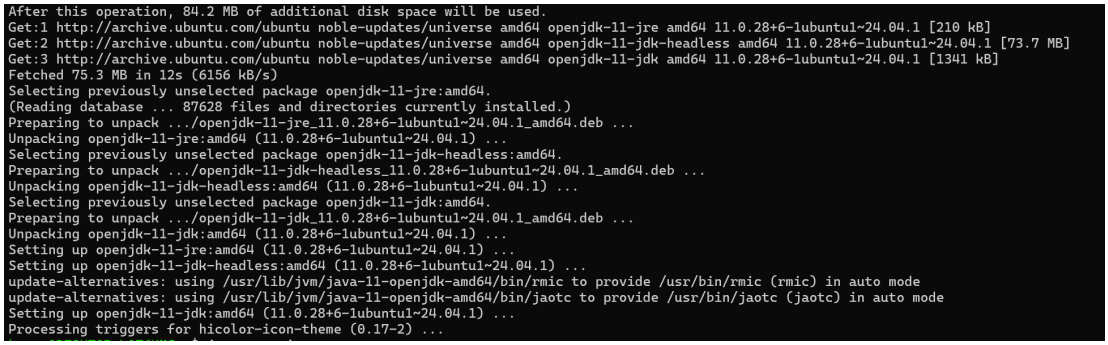
Once the update is complete, we proceed with inputting the following lines to install both JDK 11 and 17: `sudo apt install openjdk-11-jdk -y` for JDK 11 and this line `sudo apt install openjdk-17-jdk -y` for JDK 17, as shown below.

Install JDK 17; *in this case, JDK 17 is already installed*:



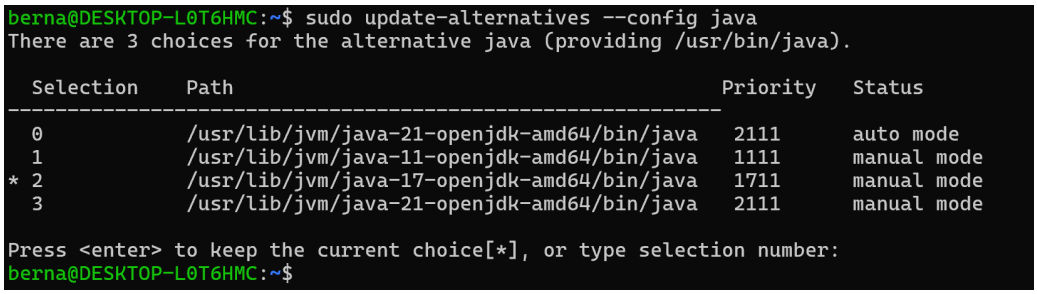
Install JDK 11:





Show Java version

After installation, we will verify that the JDK is installed and determine the default JDK being used. In the image below, it can be seen that the JDKs installed are 11 for Cassandra, 17 for Kafka and Spark, and 21, which will not be used. Using the command `sudo update-alternatives --config java` will output the Java versions installed and show that JDK 17 is the one being used by the system:



To further verify the java version, inputting the command `java --version` will output the current Java version used, which is JDK 17:

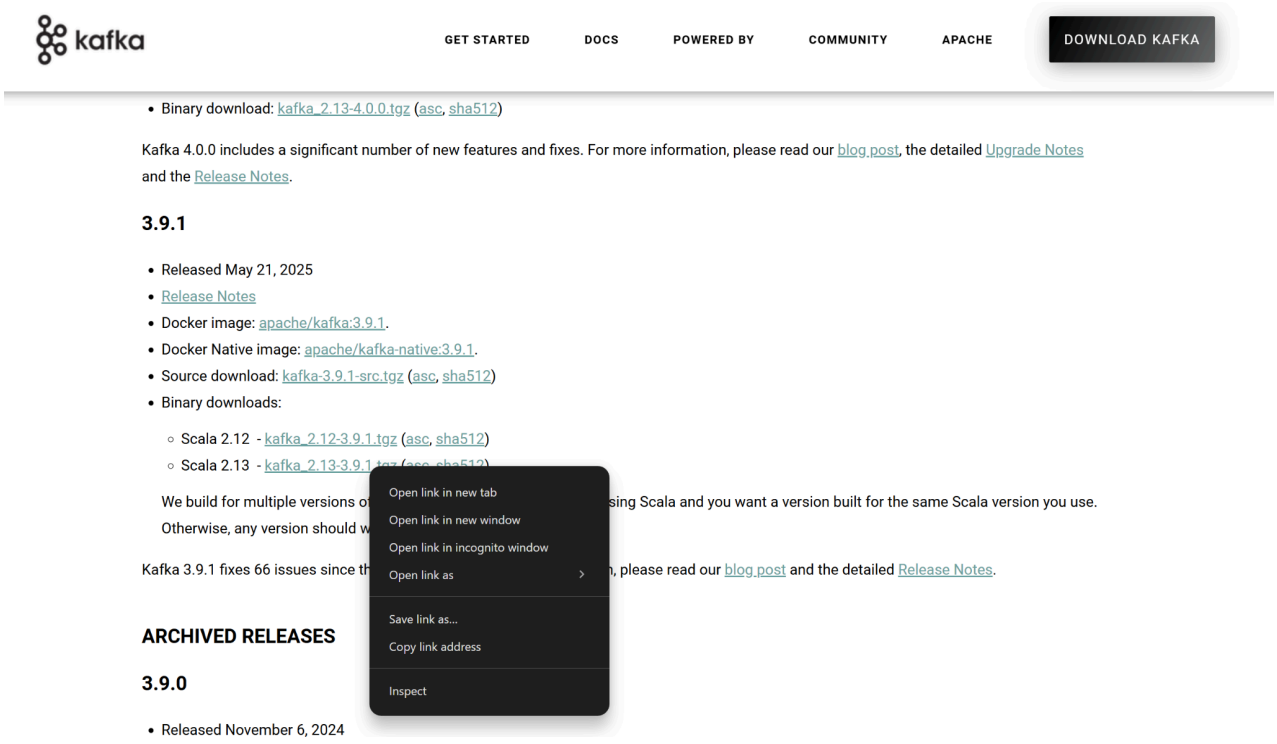
```
berna@DESKTOP-L0T6HMC:~$ java --version
openjdk 17.0.16 2025-07-15
OpenJDK Runtime Environment (build 17.0.16+8-Ubuntu-0ubuntu124.04.1)
OpenJDK 64-Bit Server VM (build 17.0.16+8-Ubuntu-0ubuntu124.04.1, mixed mode, sharing)
berna@DESKTOP-L0T6HMC:~$
```

Part 0.2 - Installing and Setting Up Kafka

After installing Java, we can now proceed to download Apache Kafka from its official website (Apache Kafka, 2025a). Going to https://kafka.apache.org/downloads, we shall select the latest NON-4.0 version. We emphasize not using versions 4.0 and above, as they no longer utilize Zookeeper (Apache Kafka, 2025b).

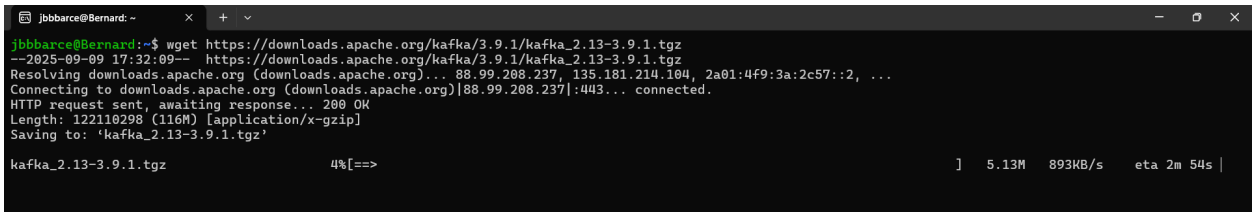
Going to Apache Kafka

In the image below, we will select the `kafka_2.13.-3.9.1.tgz` file and click the *copy link address* to download Kafka.



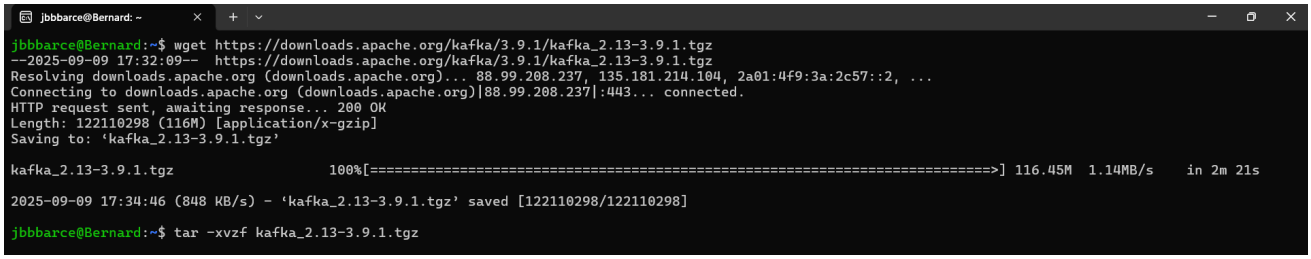
After which, we will then paste it after the keyword `wget`, which looks like this

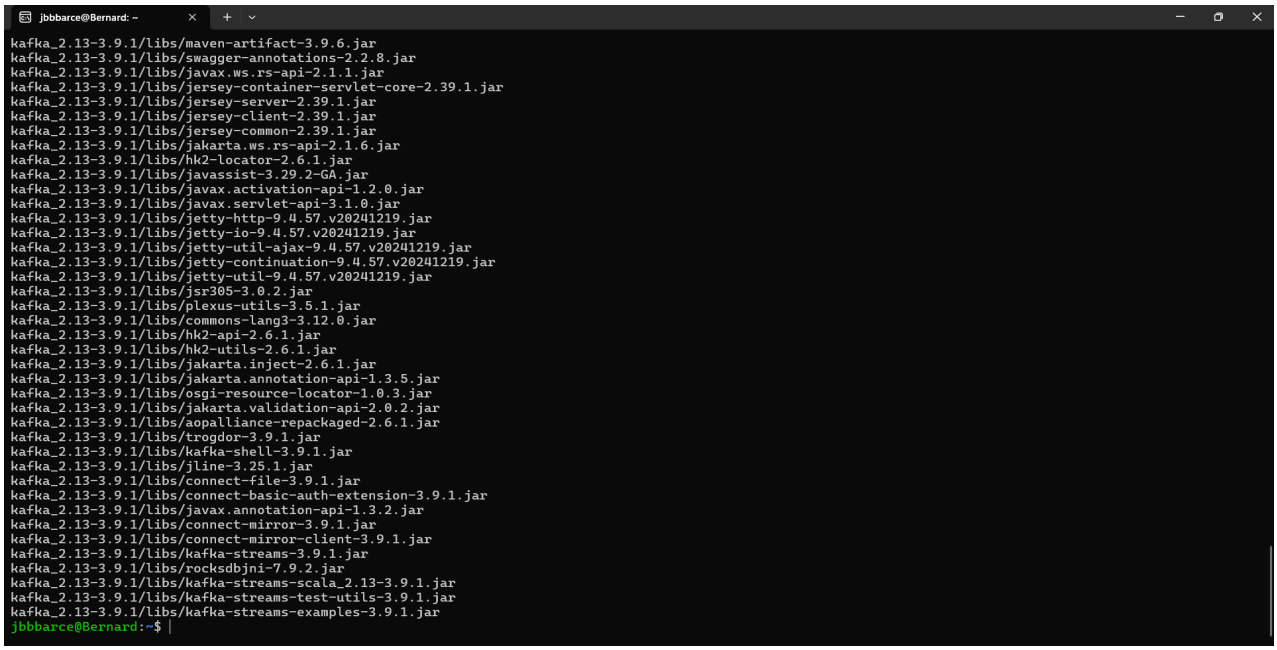
`wget https://dlcdn.apache.org/kafka/3.9.1/kafka_2.13-3.9.1.tgz`, which will then proceed to download Kafka.



Extract Kafka

After the installation is completed, we then proceed to extract it by inputting `tar -xvzf kafka_2.13-3.9.1.tgz`.





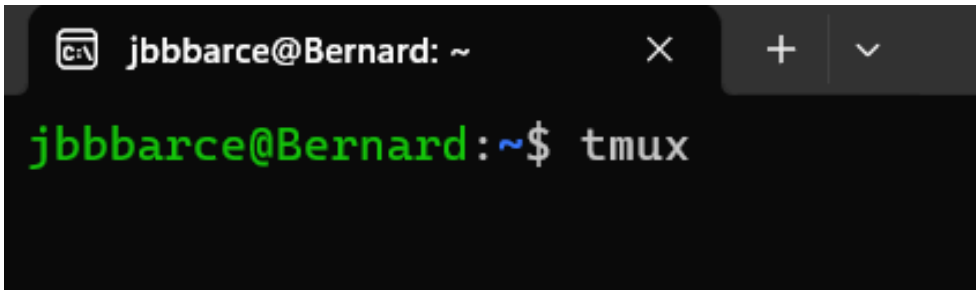
Once extracted, we now proceed to run Zookeeper and Kafka.

Starting ZooKeeper server

We first start ZooKeeper. Kafka relies on ZooKeeper for broker coordination. ZooKeeper manages cluster metadata and maintains consistency among distributed systems (ZooKeeper,

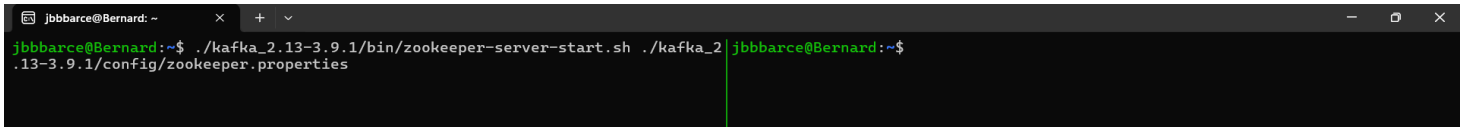
2025). That is why the Zookeeper must start first in order to form the connection and synchronization of Kafka brokers.

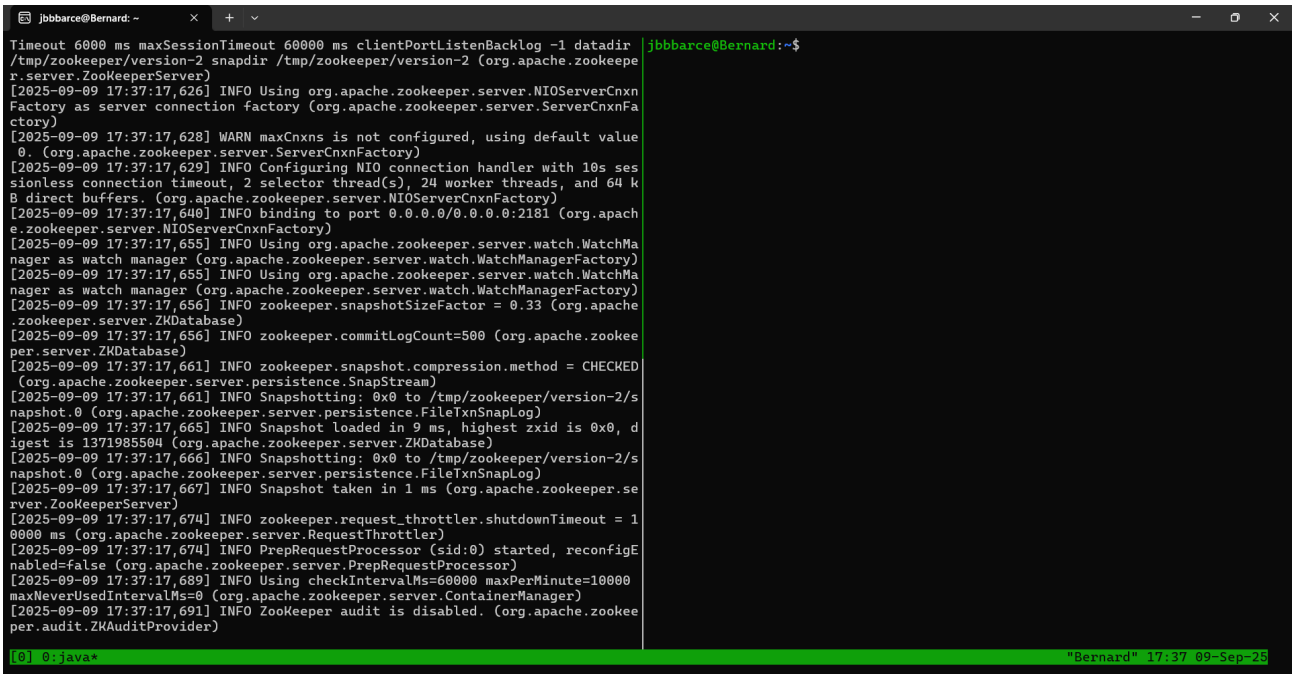
Before starting this, we should open up two terminals, one for ZooKeeper, the other for Kafka. Run `tmux`, if not yet installed, run `sudo apt install tmux`.



To open a second terminal, press *Ctrl-B* then *Shift-%* (for a vertical layout, press *Shift-")* (TmuxCheatSheet, n.d.). On one terminal, we shall start Zookeeper by inputting,

```
./kafka_2.13-3.9.1/bin/zookeeper-server-start.sh
./kafka_2.13-3.9.1/config/zookeeper.properties
```



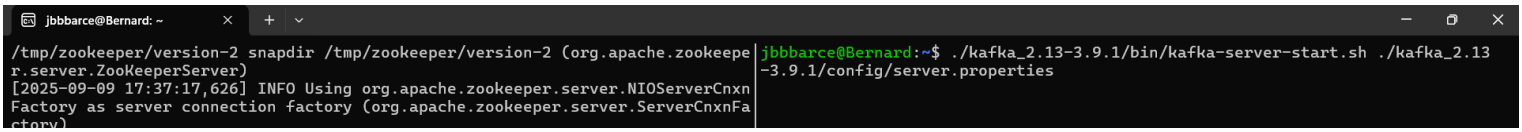


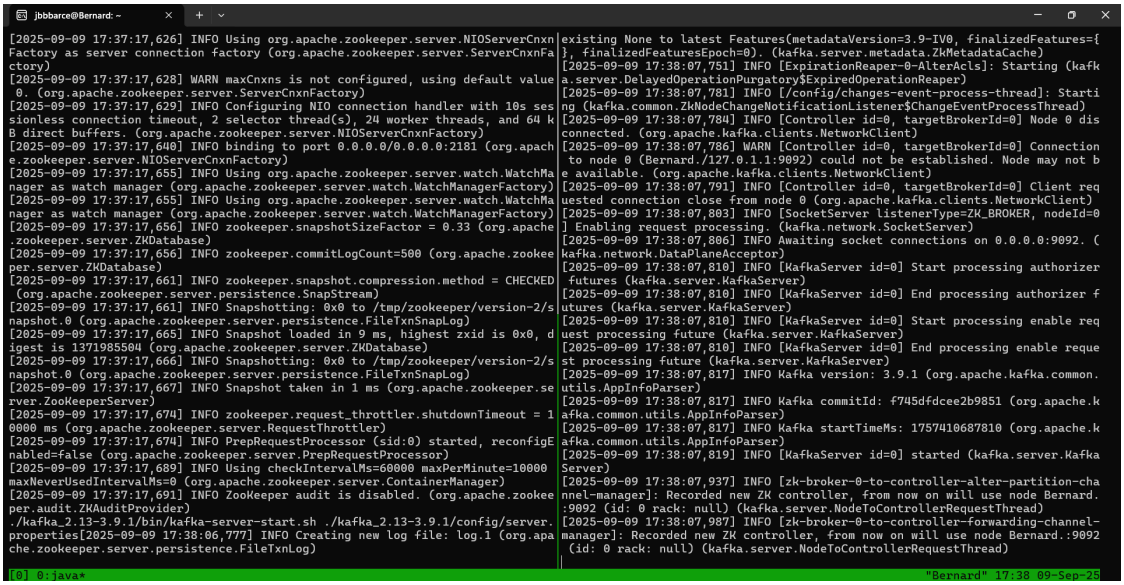
Starting the Kafka Server

Once ZooKeeper is running, we shall start Kafka; the Kafka broker handles message storage, processing, and communication with clients. Configuration settings such as ports and log directories are defined in the server properties file (Apache Kafka, 2025a).

By running this command in the second terminal:

```
./kafka_2.13-3.9.1/bin/kafka-server-start.sh
./kafka_2.13-3.9.1/config/server.properties
```



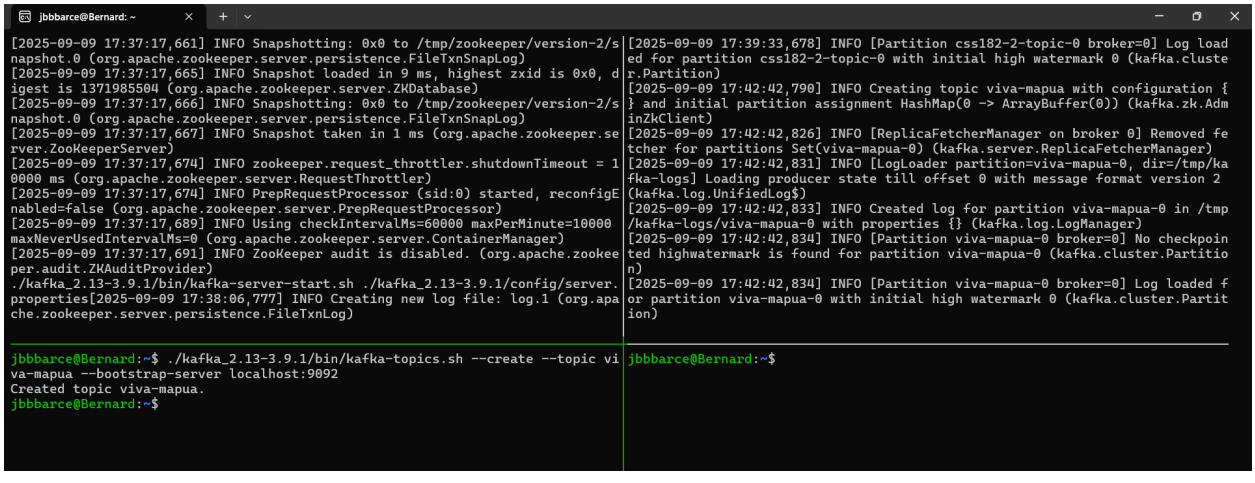


Creating a topic, producer, and consumer

After starting the Kafka server, we shall proceed to create a topic, which in this walkthrough, the topic will be *viva-mapua*, **but in the main project, a different topic will be use**d.

We added another terminal using tmux commands, and then proceeded to create the topic *viva-mapua* by inputting

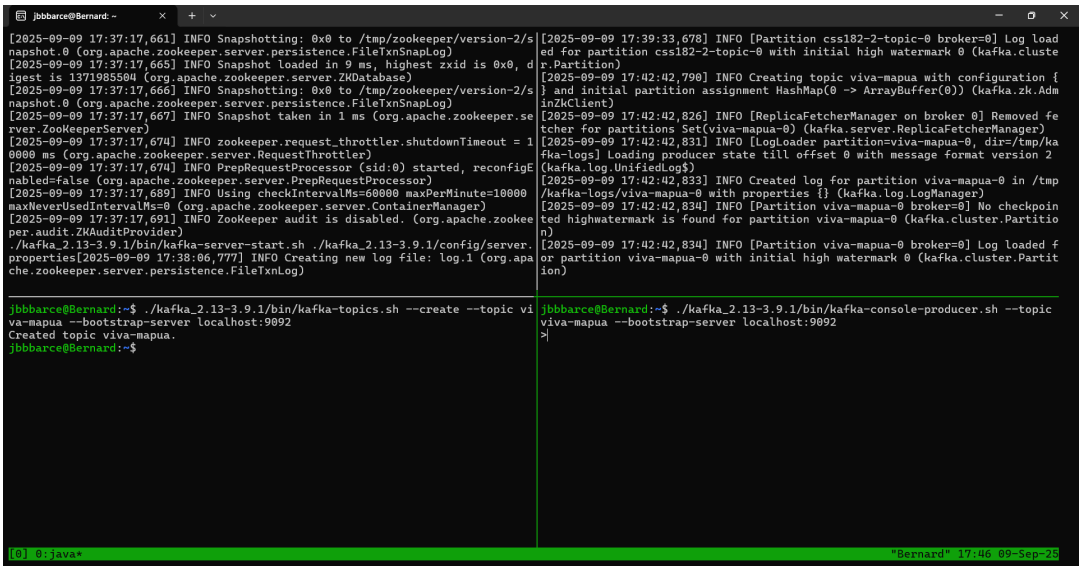
```
./kafka_2.13-3.9.1/bin/kafka-topics.sh --create --topic viva-mapua
--bootstrap-server localhost:9092
```



After creating the topic, we then proceed to create the producer, which will be the one responsible for writing data to the topic. Take note that in the line of code we always specify the topic.
By inputting:

```
./kafka_2.13-3.9.1/bin/kafka-console-producer.sh --topic viva-mapua
--bootstrap-server localhost:9092
```

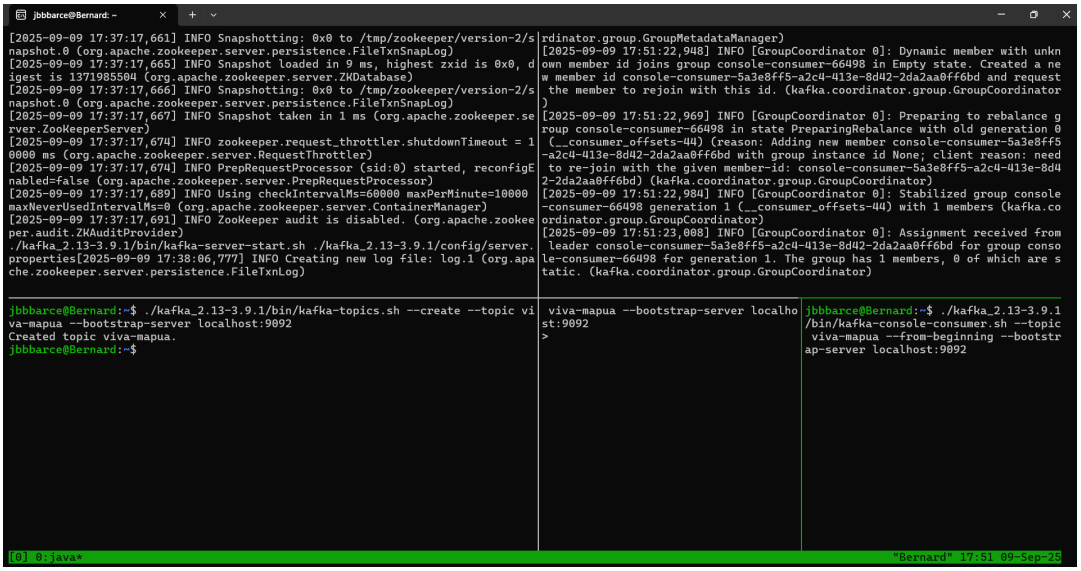
We proceed in creating the producer:



After creating the producer, we then create the consumer, which will receive or read the messages from the topic created by the producer.
By inputting:

```
./kafka_2.13-3.9.1/bin/kafka-console-consumer.sh --topic viva-mapua
--from-beginning --bootstrap-server localhost:9092
```

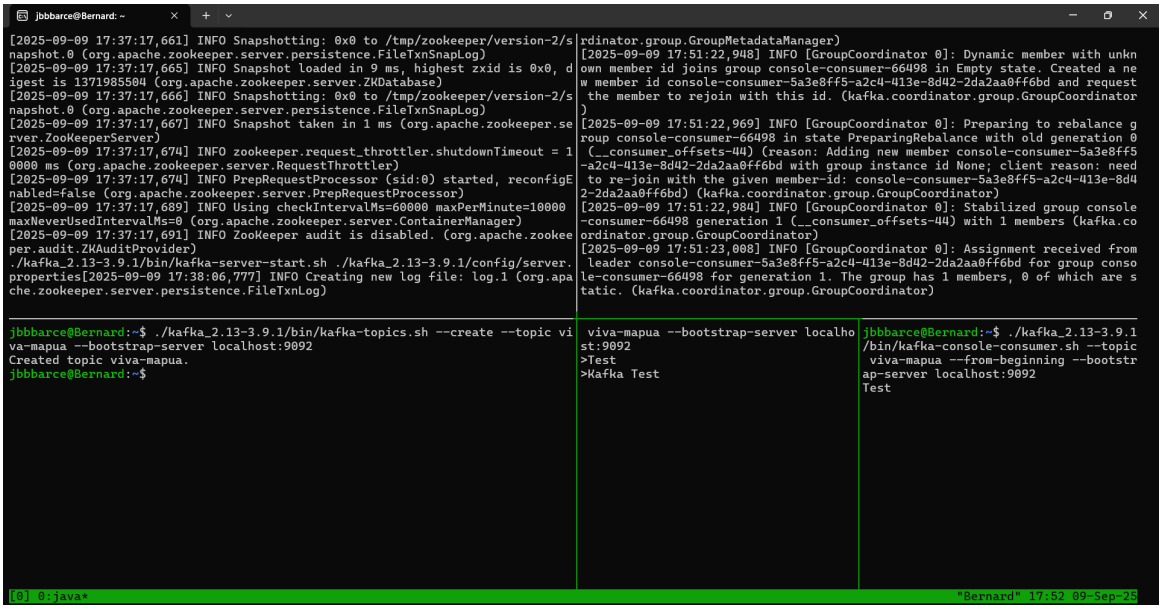
We proceed in creating the consumer:



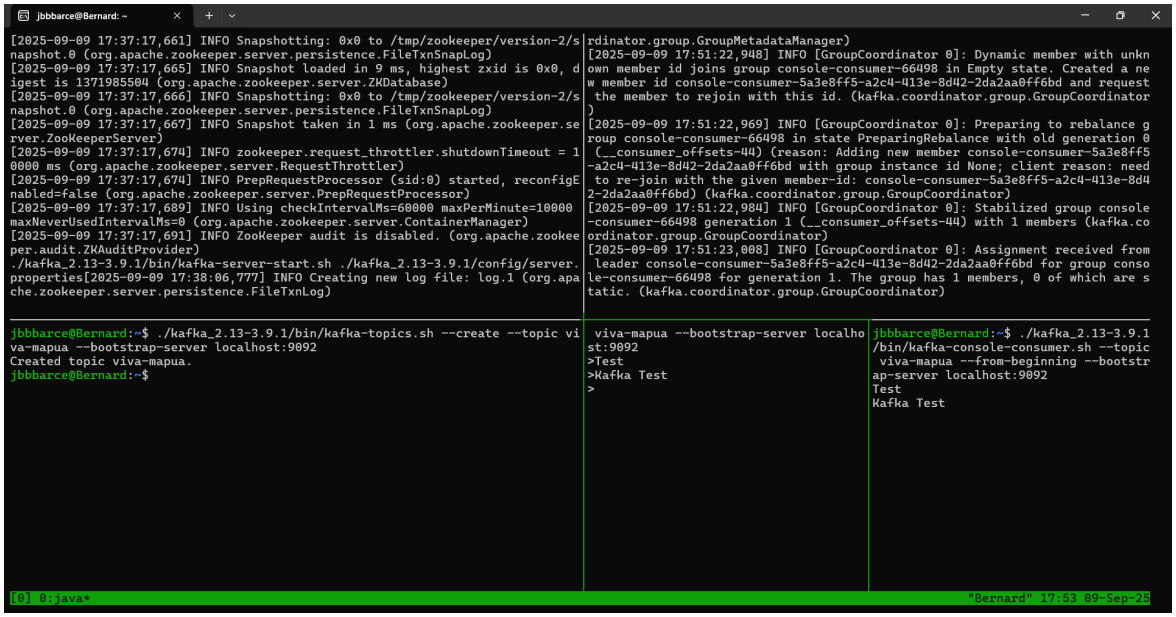
Test

Lastly, we now test. Testing involves sending a message through the producer and verifying if it appeared in the consumer terminal. If the message is received, the Kafka installation and configuration are confirmed successful

Input test messages in the consumer.



The message from the producer appeared in the consumer, indicating that the installation and setup of Kafka were successful.



Now that we have set up and installed Kafka, we then proceed to install Spark. You can now close the terminals and open a new one for the Spark installation.

Part 0.3 - Installing and Setting Up Spark

This guide will discuss how to install and set up Apache Spark if you have not already done so. Assuming you wanted to install Spark first, before installing input `sudo apt update`, to have the latest packages and metadata installed, but if you have recently done this command, proceed to the next step.

Installing and Extract Spark

To install Spark, we use this command:

```
wget https://dlcdn.apache.org/spark/spark-4.0.1/spark-4.0.1-bin-hadoop3.tgz
```

```
berna@DESKTOP-L0T6HMC:~$ wget https://downloads.apache.org/spark/spark-4.0.1/spark-4.0.1-bin-hadoop3.tgz
--2025-11-10 23:07:10--  https://downloads.apache.org/spark/spark-4.0.1/spark-4.0.1-bin-hadoop3.tgz
Resolving downloads.apache.org (downloads.apache.org)... 135.181.214.104, 88.99.208.237, 2a01:4f9:3a:2c57::2, ...
Connecting to downloads.apache.org (downloads.apache.org)|135.181.214.104|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 548955321 (524M) [application/x-gzip]
Saving to: ‘spark-4.0.1-bin-hadoop3.tgz’

spark-4.0.1-bin-hadoop3.tgz    21%[=========>                                        ] 110.28M  5.50MB/s    eta 2m 6s
```

After downloading, make sure to run

```
tar -xzvf spark-4.0.1-bin-hadoop3.tgz
sudo mv spark-4.0.1-bin-hadoop3 /opt/spark
```

This would extract the tgz file (GeeksforGeeks, 2017) and then move the created directory to /opt/spark. NOTE: moving it to /opt/spark is optional, but it is easier to locate it this way. To verify, go to the created directory, `cd /opt/spark`:

```
berna@DESKTOP-L0T6HMC:~$ cd /opt/spark
berna@DESKTOP-L0T6HMC:/opt/spark$ ls
LICENSE  R          RELEASE  conf  examples      jars        licenses  python  work
NOTICE   README.md  bin      data  hive-jackson  kubernetes  logs      sbin    yarn
berna@DESKTOP-L0T6HMC:/opt/spark$
```

Setup Spark

You have now downloaded Apache Spark and organized its directory; now, we will try to set it up. First, we will need to set the environment path. We can do so by opening the .bashrc file. Here is how you can do it:

Input `nano ~/.bashrc`, then add these lines at the bottom:

```
export SPARK_HOME=/opt/spark
export PATH=$PATH:$SPARK_HOME/bin:$SPARK_HOME/sbin
```

```
berna@DESKTOP-L0T6HMC: ·
  GNU nano 7.2                                  /home/berna/.bashrc
fi

# enable programmable completion features (you don't need to enable
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile
# sources /etc/bash.bashrc).
if ! shopt -oq posix; then
  if [ -f /usr/share/bash-completion/bash_completion ]; then
    . /usr/share/bash-completion/bash_completion
  elif [ -f /etc/bash_completion ]; then
    . /etc/bash_completion
  fi
fi

export JAVA_HOME="/usr/lib/jvm/java-17-openjdk-amd64"
export SPARK_HOME=/opt/spark
export PATH=$PATH:$SPARK_HOME/bin:$SPARK_HOME/sbin
```

After opening it using nano, navigating below, and writing Spark's path, the end result would look something like the image above. Press Ctrl+O to save what you wrote. Press enter to confirm the filename, then Ctrl-X to exit the .bashrc file. Don't forget the `source ~/.bashrc` command after closing (Ask Ubuntu, 2020).

Create a Dedicated Spark User

Next, we will attempt to create a user and verify its success. We will run the commands below:

```
sudo useradd spark
cat /etc/passwd
```

This will first create a user. Here, we used the useradd keyword to create one. The line below allows you to view the users that have been created. You will see something like this after executing the cat /etc/passwd

```
spark:x:1001:1001::/home/spark:/bin/sh
```

You should see the created user; in this case, we created the "spark" user. Next, using the commands:

```
sudo chown -R spark:spark /opt/spark
ls -la /opt/
```

This changes the rights of the /opt/spark directory to user=spark and group=spark, indicated by spark:spark (Ludosanu, 2024). Run the second line to verify the ownership.

```
berna@DESKTOP-L0T6HMC:~$ ls -la /opt/
total 12
drwxr-xr-x  3 root  root  4096 Sep 18 21:28 .
drwxr-xr-x 27 root  root  4096 Nov 10 12:51 ..
drwxr-xr-x 16 spark spark 4096 Sep 18 21:34 spark
```

Create systemd Service Files

Before we attempt to execute and start Spark, we need to configure the spark-master.service and spark-worker.service for Spark 4.0.1. Both of these are located at /sbin, but we will input this command for the master first:

```
sudo nano /etc/systemd/system/spark-master.service
```

Once inside, input these codes, and Ctrl+O to save and Ctrl+x to exit.:

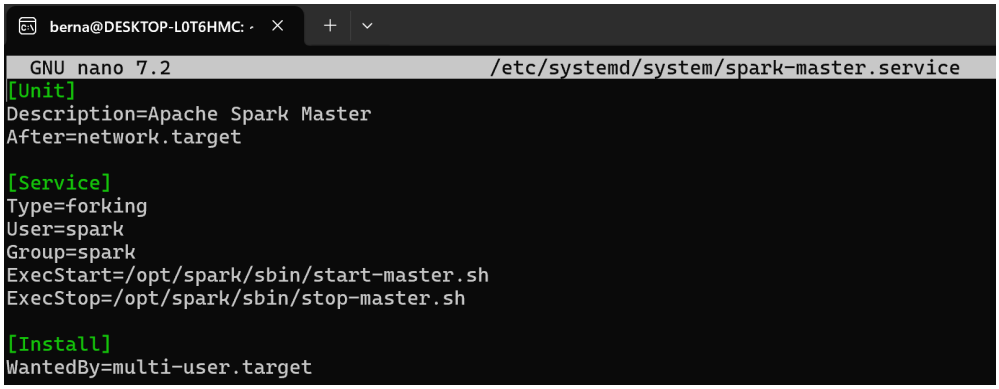
```
[Unit]
Description=Apache Spark Master
After=network.target

[Service]
Type=forking
```

```
User=spark
Group=spark
ExecStart=/opt/spark/sbin/start-master.sh
ExecStop=/opt/spark/sbin/stop-master.sh

[Install]
WantedBy=multi-user.target
```

Just like this:



Do the same thing for the spark-worker. Below is the needed syntax and corresponding result:

```
sudo nano /etc/systemd/system/spark-worker.service
```

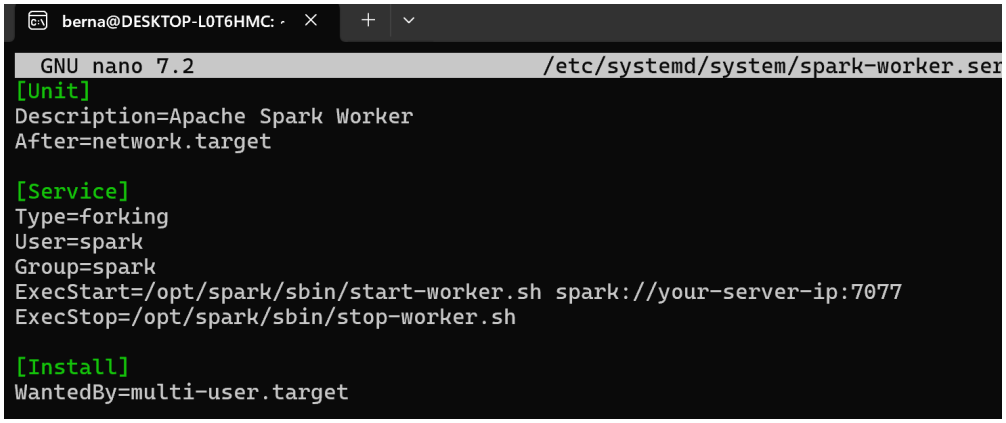
Once inside, input these codes, and Ctrl+O to save and Ctrl+x to exit:

```
[Unit]
Description=Apache Spark Master
After=network.target

[Service]
Type=forking
User=spark
Group=spark
ExecStart=/opt/spark/sbin/start-master.sh
ExecStop=/opt/spark/sbin/stop-master.sh

[Install]
WantedBy=multi-user.target
```

Just like this:



Start Spark to test

After that, we now proceed to start Spark by sequentially inputting these commands:

```
sudo systemctl daemon-reload
sudo systemctl enable spark-master
sudo systemctl enable spark-worker
sudo systemctl start spark-master
sudo systemctl start spark-worker
```

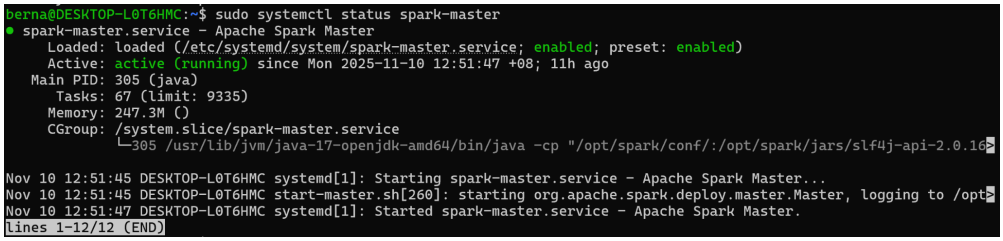
Just like this:

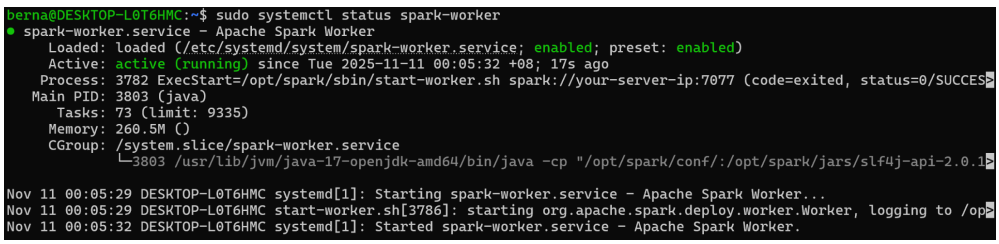
```
berna@DESKTOP-L0T6HMC:~$ sudo systemctl daemon-reload
berna@DESKTOP-L0T6HMC:~$ sudo systemctl enable spark-master
berna@DESKTOP-L0T6HMC:~$ sudo systemctl enable spark-worker
berna@DESKTOP-L0T6HMC:~$ sudo systemctl start spark-master
berna@DESKTOP-L0T6HMC:~$ sudo systemctl start spark-worker
```

After that, we check the status for both spark-master and spark-worker using these commands:

```
sudo systemctl status spark-master
sudo systemctl status spark-worker
```

This should be the outputs:





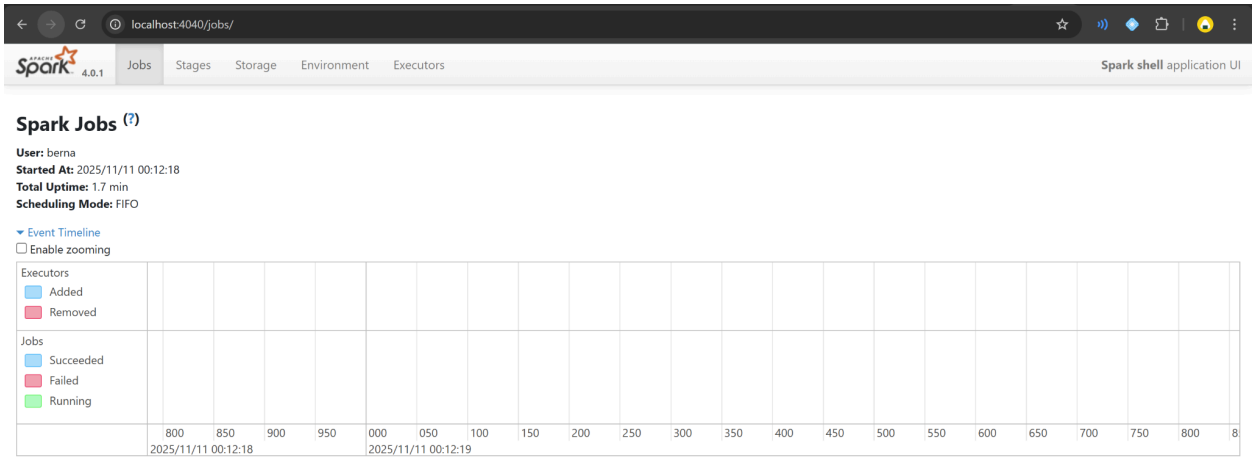
Lastly, Run this command to start your spark-shell:`spark-shell`. You would see this output after running and loading:

```
berna@DESKTOP-L0T6HMC:~$ spark-shell
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
25/11/11 00:12:14 WARN Utils: Your hostname, DESKTOP-L0T6HMC, resolves to a loopback address: 127.0.1.1; using 10.255.255.254 instead (on interface lo)
25/11/11 00:12:14 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 4.0.1
      /_/

Using Scala version 2.13.16 (OpenJDK 64-Bit Server VM, Java 17.0.16)
Type in expressions to have them evaluated.
Type :help for more information.
25/11/11 00:12:18 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Spark context Web UI available at http://10.255.255.254:4040
Spark context available as 'sc' (master = local[*], app id = local-1762791139715).
Spark session available as 'spark'.

scala>
```

After following through these steps, you'd have successfully set up your Apache Spark. Go to your browser and type localhost:4040. You will now be able to access the Spark shell application UI as seen below.



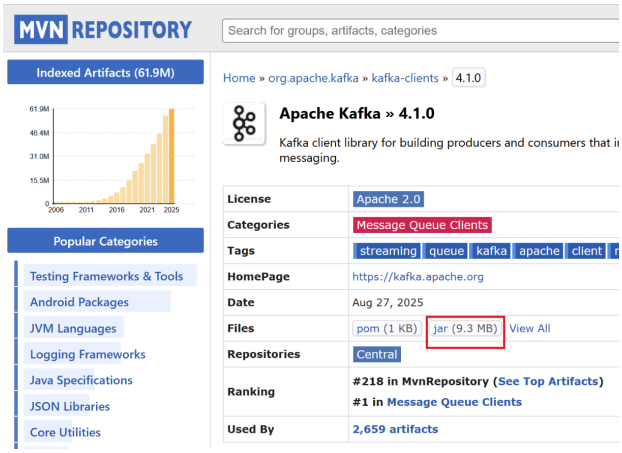
Spark-Kafka Integration

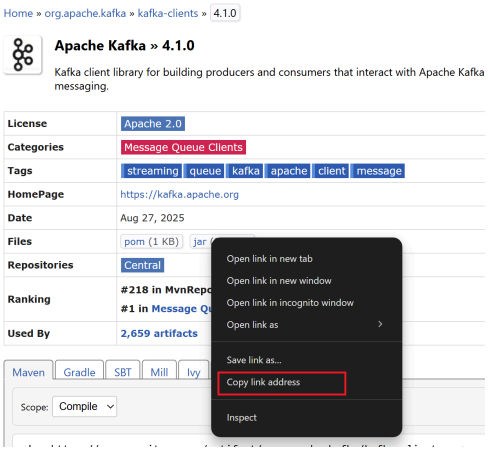
Now that Spark is installed and ready, it is also needs to be integrated with Kafka for this project. In that context, we must integrate by first installing the needed jar files for this integration inside `/opt/spark/`.

We then go to Maven repositories and search for these jar files; the jar files depend on your Spark version. Since our Spark version is 4.0.1, these are the jar files needed for compatibility:

```
kafka-clients-4.1.0.jar
spark-streaming-kafka-0-10_2.13-4.0.1.jar
spark-sql-kafka-0-10_2.13-4.0.1.jar
spark-streaming-kafka-0-10-assembly_2.13-4.0.1.jar
commons-pool2-2.12.1.jar
```

Go to Maven repositories and search for the jar files mentioned above. Alternatively, if you are using a specific Spark version, right-click the jar and then press 'Copy Link Address'. Do the same for every jar file needed; an example is shown below:

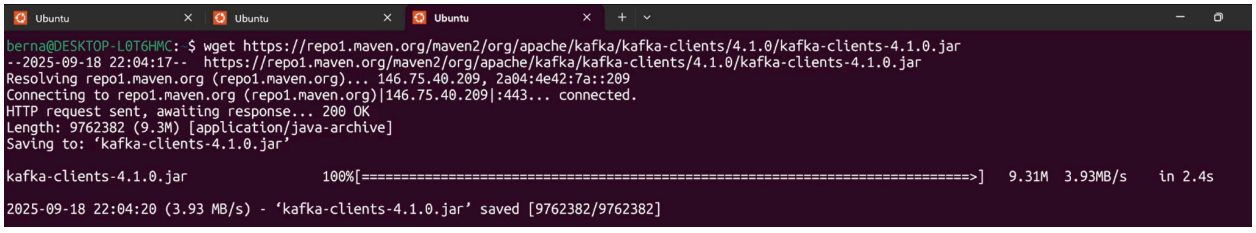




Also, here are the links to the jar files. The version is as stated above:

- https://repo1.maven.org/maven2/org/apache/kafka/kafka-clients/4.1.0/kafka-clients-4.1.0.jar
- https://repo1.maven.org/maven2/org/apache/spark/spark-streaming-kafka-0-10_2.13/4.0.1/spark-streaming-kafka-0-10_2.13-4.0.1.jar
- https://repo1.maven.org/maven2/org/apache/spark/spark-sql-kafka-0-10_2.13/4.0.1/spark-sql-kafka-0-10_2.13-4.0.1.jar
- https://repo1.maven.org/maven2/org/apache/spark/spark-streaming-kafka-0-10-assembly_2.13/4.0.1/spark-streaming-kafka-0-10-assembly_2.13-4.0.1.jar
- https://repo1.maven.org/maven2/org/apache/commons/commons-pool2/2.12.1/commons-pool2-2.12.1.jar

Going back, after copying the link address of the correct version of the jar file, go back to the WSL terminal and paste it beside `wget` for it to be installed.



After the jar is installed, move it to where the jar files of Spark are downloaded; in this case, it is in `/opt/spark/jars`.

So proceed with `sudo mv [jar file name] /opt/spark/jars/`



And then proceed to check using keywords in the jar file name. You can use the `grep` keyword to specifically look for exact matches.

```
berna@DESKTOP-L0T6HMC:~$ ls -1 /opt/spark/jars |  grep kafka
kafka-clients-4.1.0.jar
```

Note: Repeat this process for all needed jar files. Once finished and all jar files are confirmed inside the correct directory, proceed to the next step. Installing these jar files is required before proceeding.

Start Spark and Import the needed libraries

After installing the required jar files, proceed to start Spark by typing this command inside `/opt/spark/`. The reason why it must be inside `/opt/spark/` is that it is the directory that contains both the startup scripts and the added JARs needed for Kafka integration.

Inside `/opt/spark/` input this line,

```
./bin/spark-shell --jars jars/kafka-clients-4.1.0.jar,
jars/spark-streaming-kafka-0-10_2.13-4.0.1.jar,
jars/spark-sql-kafka-0-10_2.13-4.0.1.jar,
jars/spark-streaming-kafka-0-10-assembly_2.13-4.0.1.jar,
jars/commons-pool2-2.12.1.jar
```

This should be the output:

```
berna@DESKTOP-L0T6HMC:~$ cd /opt/spark
berna@DESKTOP-L0T6HMC:/opt/spark$ ./bin/spark-shell --jars jars/kafka-clients-4.1.0.jar, jars/spark-streaming-kafka-0-10_2.13-4.0.1.jar, jars/spark-sql-kafka-
0-10_2.13-4.0.1.jar, jars/spark-streaming-kafka-0-10-assembly_2.13-4.0.1.jar, jars/commons-pool2-2.12.1.jar
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
25/11/11 00:36:47 WARN Utils: Your hostname, DESKTOP-L0T6HMC, resolves to a loopback address: 127.0.1.1; using 10.255.255.254 instead (on interface lo)
25/11/11 00:36:47 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
25/11/11 00:36:47 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 4.0.1
      /_/

Using Scala version 2.13.16 (OpenJDK 64-Bit Server VM, Java 17.0.16)
Type in expressions to have them evaluated.
Type :help for more information.
Spark context Web UI available at http://10.255.255.254:4040
Spark context available as 'sc' (master = local[*], app id = local-1762792612882).
Spark session available as 'spark'.

scala>
```

We are now done setting up Spark; now we proceed with installing and setting up Apache Cassandra.

Part 0.3: Install and Configure Apache Cassandra

After Kafka and Spark, we proceed to the installation and configuration of Apache Cassandra, you can skip this part, if you have already Cassandra. Moving on, there are two methods: one is the APT repository method (which will only be discussed), and the other is using the wget command to extract the tarball from the Apache Archives, which will be the method used in this guide.

APT Method

The APT method involves first installing prerequisites, such as GnuPG2, curl, and apt-transport-https, to ensure secure software installations by verifying package signatures. GnuPG2 provides the tools for APT to check a repository's release files against the publisher's GPG key, preventing the installation of tampered packages (Gite, 2025). Following this, the Apache Cassandra GPG key is imported, and the official APT repository is added, allowing APT to securely install the latest vendor builds and updates from Apache (Apache Cassandra, n.d.-a).

Install Cassandra using wget and extract the tar.gz

We install Apache Cassandra using the wget command. In this case, version 4.0.18 was retrieved from the Apache Archives, and then we proceeded to copy its link address and install the tarball (.tar.gz) package, which contains the precise build for Cassandra, and install it using wget using this command:

`wget https://archive.apache.org/dist/cassandra/4.0.18/apache-cassandra-4.0.18-bin.tar.gz`

```
berna@DESKTOP-L0T6HMC:~$ wget https://archive.apache.org/dist/cassandra/4.0.18/apache-cassandra-4.0.18-bin.tar.gz
--2025-10-26 20:40:37--  https://archive.apache.org/dist/cassandra/4.0.18/apache-cassandra-4.0.18-bin.tar.gz
Resolving archive.apache.org (archive.apache.org)... 65.108.204.189, 2a01:4f9:1a:a084::2
Connecting to archive.apache.org (archive.apache.org)|65.108.204.189|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 49788954 (47M) [application/x-gzip]
Saving to: 'apache-cassandra-4.0.18-bin.tar.gz'

apache-cassandra-4.0.18-bin.tar 100%[=====================================================>]  47.48M  2.59MB/s    in 19s

2025-10-26 20:40:56 (2.48 MB/s) - 'apache-cassandra-4.0.18-bin.tar.gz' saved [49788954/49788954]
```

What this is: A compressed tarball that contains Cassandra's **binaries** (`bin/`), **libraries** (`lib/`), **configuration** (`conf/`), and **tools** (`tools/`).

Extract the tar

After that, we will extract the tarball downloaded using the following command: `tar -zxvf apache-cassandra-4.0.18-bin.tar.gz`. In order to decompress and unpack the full Cassandra directory in place, so the launcher scripts can find their expected layout.

```
berna@DESKTOP-L0T6HMC:~$ tar -zxvf apache-cassandra-4.0.18-bin.tar.gz
apache-cassandra-4.0.18/bin/
apache-cassandra-4.0.18/conf/
apache-cassandra-4.0.18/conf/triggers/
apache-cassandra-4.0.18/doc/
apache-cassandra-4.0.18/doc/cql3/
apache-cassandra-4.0.18/lib/
apache-cassandra-4.0.18/lib/sigar-bin/
apache-cassandra-4.0.18/pylib/
apache-cassandra-4.0.18/pylib/cqlshlib/
apache-cassandra-4.0.18/pylib/cqlshlib/test/
```

Then we proceed to move the extracted tarball into a new Cassandra folder to keep the installation clean and easy to reference from the home directory using the command: `mv apache-cassandra-4.0.18 Cassandra`.

```
berna@DESKTOP-L0T6HMC:~$
berna@DESKTOP-L0T6HMC:~$ mv apache-cassandra-4.0.18 Cassandra
```

Create the Startup Script

We then proceed to create the Script that was mentioned earlier, since this Cassandra version uses JDK 11, while other apps, such as Kafka and Spark, utilize JDK 17, so in order not to avoid version conflicts, we will create a bash script called run-cassandra.sh.sh in order to set the jdk 11 as the one to be used by Cassandra when running.

Using the command `nano ~/Cassandra/run-cassandra.sh` will open the nano text editor:

```
berna@DESKTOP-L0T6HMC:~$ nano ~/Cassandra/run-cassandra.sh
```

The startup script will look like this, in which it specifies that the startup for Cassandra will utilize JDK 11. After creating the script, save it using Ctrl-O, and exit the editor using Ctrl-X.

```
  GNU nano 7.2                                    run-cassandra.sh
#!/bin/bash

export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
export PATH=$JAVA_HOME/bin:$PATH

cd ~/Cassandra
bin/cassandra
```

We then make the script executable using `chmod +x ~/Cassandra/run-cassandra.sh`

```
berna@DESKTOP-L0T6HMC:~$ chmod +x ~/Cassandra/run-cassandra.sh
```

Note: if your default Java is JDK 11, there is no need to create a script, after doing the mv apache-cassandra-4.0.18 Cassandra command. You can simply proceed into your environment (if you have one) or main directory (cd ~) and input this command in order to run Cassandra `cd ~/Cassandra && bin/cassandra`.

Run script

Once the script is made, we will go inside an environment (optional) and go inside the Cassandra directory, and start the server by running the script.

The command will be if you have an environment, in this case:

`source cassandra_env/bin/activate` => `cd Cassandra` => `bash run-cassandra.sh`

```
berna@DESKTOP-L0T6HMC:~$
berna@DESKTOP-L0T6HMC:~$ source cassandra_env/bin/activate
(cassandra_env) berna@DESKTOP-L0T6HMC:~$ cd Cassandra
(cassandra_env) berna@DESKTOP-L0T6HMC:~/Cassandra$ ls
CASSANDRA-14092.txt  LICENSE.txt  NOTICE.txt  conf  doc  logs   run-cassandra.sh
CHANGES.txt          NEWS.txt     bin         data  lib  pylib  tools
(cassandra_env) berna@DESKTOP-L0T6HMC:~/Cassandra$ bash run-cassandra.sh
(cassandra_env) berna@DESKTOP-L0T6HMC:~/Cassandra$ OpenJDK 64-Bit Server VM warning: Option UseConcMarkSweepGC was depre
cated in version 9.0 and will likely be removed in a future release.
CompileCommand: dontinline org/apache/cassandra/db/Columns$Serializer.deserializeLargeSubset(Lorg/apache/cassandra/io/ut
il/DataInputPlus;Lorg/apache/cassandra/db/Columns;I)Lorg/apache/cassandra/db/Columns;
CompileCommand: dontinline org/apache/cassandra/db/Columns$Serializer.serializeLargeSubset(Ljava/util/Collection;ILorg/a
pache/cassandra/db/Columns;ILorg/apache/cassandra/io/util/DataOutputPlus;)V
CompileCommand: dontinline org/apache/cassandra/db/Columns$Serializer.serializeLargeSubsetSize(Ljava/util/Collection;ILo
rg/apache/cassandra/db/Columns;I)I
CompileCommand: dontinline org/apache/cassandra/db/commitlog/AbstractCommitLogSegmentManager.advanceAllocatingFrom(Lorg/
apache/cassandra/db/commitlog/CommitLogSegment;)V
CompileCommand: dontinline org/apache/cassandra/db/transform/BaseIterator.tryGetMoreContents()Z
CompileCommand: dontinline org/apache/cassandra/db/transform/StoppingTransformation.stop()V
CompileCommand: dontinline org/apache/cassandra/db/transform/StoppingTransformation.stopInPartition()V
```

After running the script, open a new terminal, navigate to the Cassandra directory again, and then enter bin/nodetool status to view the current state of Cassandra. This includes details about its operational status and data distribution, verifying that the server is running successfully.

Go inside to the Cassandra directory, `cd Cassandra` , then run `bin/nodetool status`.

```
berna@DESKTOP-L0T6HMC: ~ | Ubuntu | Ubuntu

berna@DESKTOP-L0T6HMC:~$ cd Cassandra
berna@DESKTOP-L0T6HMC:~/Cassandra$ bin/nodetool status
Datacenter: datacenter1
=======================
Status=Up/Down
|/ State=Normal/Leaving/Joining/Moving
--  Address    Load        Tokens  Owns (effective)  Host ID                               Rack
UN  127.0.0.1  198.35 KiB  16      100.0%            3036802d-3848-4e1d-94d3-422f876f05f6  rack1
```

Then, on the same terminal, we will input `cqlsh` or Cassandra Query Language shell. The cqlsh is an interactive client that connects to a Cassandra node, allowing you to run CQL statements, create keyspaces/tables, insert and query data, and perform basic administrative tasks such as inspecting schemas and cluster status.

Note: In some cases, you need to install cqlsh first using this command: `sudo snap install cqlsh`*. After installing it, you can now run* `cqlsh`*.*

The output should look like this, in which you can see that Cassandra is connected to a cluster and the cqlsh query will appear:

```
berna@DESKTOP-L0T6HMC:~/Cassandra$ cqlsh
Connected to Test Cluster at 127.0.0.1:9042.
[cqlsh 5.0.1 | Cassandra 4.0.18 | CQL spec 3.4.5 | Native protocol v5]
Use HELP for help.
cqlsh>
```

We are now finished in setting up Cassandra, Kafka, and Spark, we now proceed in the main pipeline.

Part 1

For Part 1, before we proceed with the streaming pipeline, we need first to setup the 10 producers needed for the streaming and setup as well the table in Cassandra in where data will be streamed.

Note: Before proceeding we recommend to setup a directory for the main project, a directory for the python code to fetch the data, and a python environment to run 10 producers and the consumer with the same python packages.

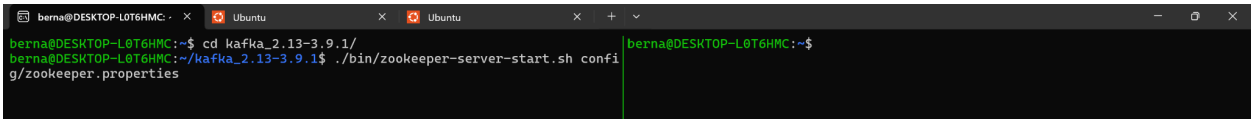
Start zookeeper and kafka

So to begin with this pipeline, we begin by starting zookeeper first since this version of Kafka still utilizes zookeeper (Apache Kafka, 2025). We begin by going inside the directory of Kafka and then run Zookeeper. *(Note: we use tmux to add a terminal, but you can also do it in a separate tab for a new terminal).*

The code to start zookeper:

```
 cd kafka_2.13-3.9.1/
./bin/zookeeper-server-start.sh config/zookeeper.properties
```

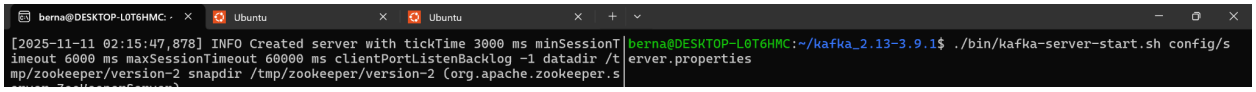
Just like this:



Now that Zookeeper is started, we now proceed in starting Kafka, using the same method of going inside the directory of Kafka and then inputting the code for its startup:

```
 cd kafka_2.13-3.9.1/
./bin/kafka-server-start.sh config/server.properties
```

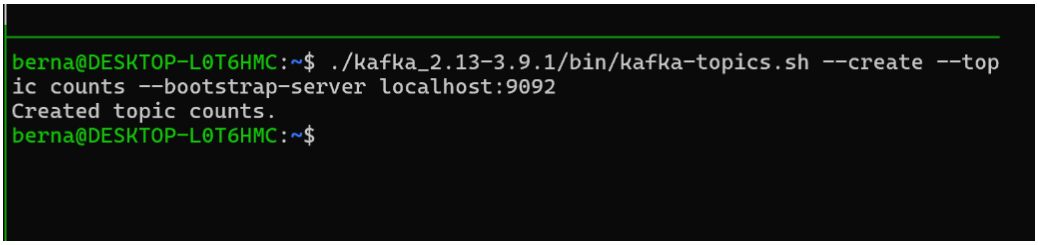
Just like this:



Create Topic

Now that Zookeeper and Kafka is up and running, we then proceed to create a topic for the producers (Apache Kafka, 2025). We added a third terminal in tmux and inputted this code to create the topic:

```
./kafka_2.13-3.9.1/bin/kafka-topics.sh --create --topic counts
--bootstrap-server localhost:9092
```



Producer and Consumer

Now that we have a topic, the producer and consumer we will be using will be inside a python code. The codes are presented below but that is not yet the final code to be used. These codes below must be inside the directory of your project, in our case its inside FinalProjDsci-2, put these codes in your project directory with the filenames `virtual_producer.py` and `virtual_consumer.py`.

Go inside your directory using cd, and using `nano virtual_producer.py`, paste code below inside it, this will create `virtual_producer.py`. Then don't forget to save using Ctrl-O then press Enter, then to exit use Ctrl-X then Enter.

Code for `virtual_producer.py`:

```
from kafka import KafkaProducer
from datetime import datetime
import time
from json import dumps
import random

import time_uuid, uuid

# pip install kafka-python
# pip install time-uuid

KAFKA_TOPIC_NAME_CONS = "counts"   #topic
KAFKA_BOOTSTRAP_SERVERS_CONS = '127.0.0.1:9092'

if __name__ == "__main__":
    print("Kafka Producer Application Started ... ")
    kafka_producer_obj =
KafkaProducer(bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS_CONS,
                                        value_serializer=lambda x:
dumps(x).encode('utf-8'))

    message_list = []
    message = None
    for i in range(3600):
        i = i + 1
        date_today = datetime.now()
        message = {}
        print("Preparing message: " + str(i))
        #event_datetime = datetime.now()
```

```
        #message["order_id"] = i
        #message["uuid_id"] = str(uuid.uuid4())

        car = random.randint(0,4)
        bus = random.randint(0,2)
        truck = random.randint(0,2)
        jeepney = random.randint(0, 2)
        bike = random.randint(0, 5)
        tryke = random.randint(0, 3)
        others = random.randint(0, 2)

        total = car + bus + truck + jeepney + bike + tryke + others

        #send data
        message["timeuuid_id"] = str(time_uuid.utctime())
        message["lgu_code"] = '1200'
        message["sensor_id"] = 'sensor_01'
        message["date_saved"] = str(date_today.strftime('%m/%d/%Y'))
        message["time_saved"] = str(date_today.strftime("%X"))
        message["total"] = total
        message["car"] = car
        message["bus"] = bus
        message["truck"] = truck
        message["jeepney"] = jeepney
        message["bike"] = bike
        message["tryke"] = tryke
        message["others"] = others


        print("Message: ", message)

        #message_list.append(message)
        kafka_producer_obj.send(KAFKA_TOPIC_NAME_CONS, message)
        time.sleep(1)  #sleep every second

    # print(message_list)

    print("Kafka Producer Application Completed. ")
```

Do the same for `virtual_consumer.py`, and don't forget to save it:

```
# -*- coding: utf-8 -*-
from kafka import KafkaConsumer

consumer = KafkaConsumer(
    'counts',
    bootstrap_servers = ['127.0.0.1:9092'],
    auto_offset_reset = 'latest',
    enable_auto_commit = True
    )

for message in consumer:
    message = message.value
    print(message)
```

Once done, you can check it inside your directory by inputting `ls`:

```
berna@DESKTOP-L0T6HMC:~$ cd FinalProjDsci-2
berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ ls
virtual_consumer.py  virtual_consumer.py:Zone.Identifier  virtual_producer.py  virtual_producer.py:Zone.Identifier
```

Create 10 instances of the Producer

As mentioned, the producer above is not yet the final one to be used because we will be using the `virtual_producer.py` to create 10 instances of it and make the sensor_ids to 1-10. In order to do that, we will be creating a bash script to create the 10 instances of virtual_producer, by creating a bash script we will be able to create the 10 producers with the contents of virtual_producer.py.

This will be done inside the project directory, and using the command nano prep_producers.sh, we will create the bash script

```
berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ nano prep_producers.sh
```

Inside it, paste this code:

```
#!/bin/bash
set -e

# To make 10 copies of your uploaded producer
for n in $(seq -w 01 10); do
  cp -f virtual_producer.py "virtual_producer_sensor_${n}.py"
done

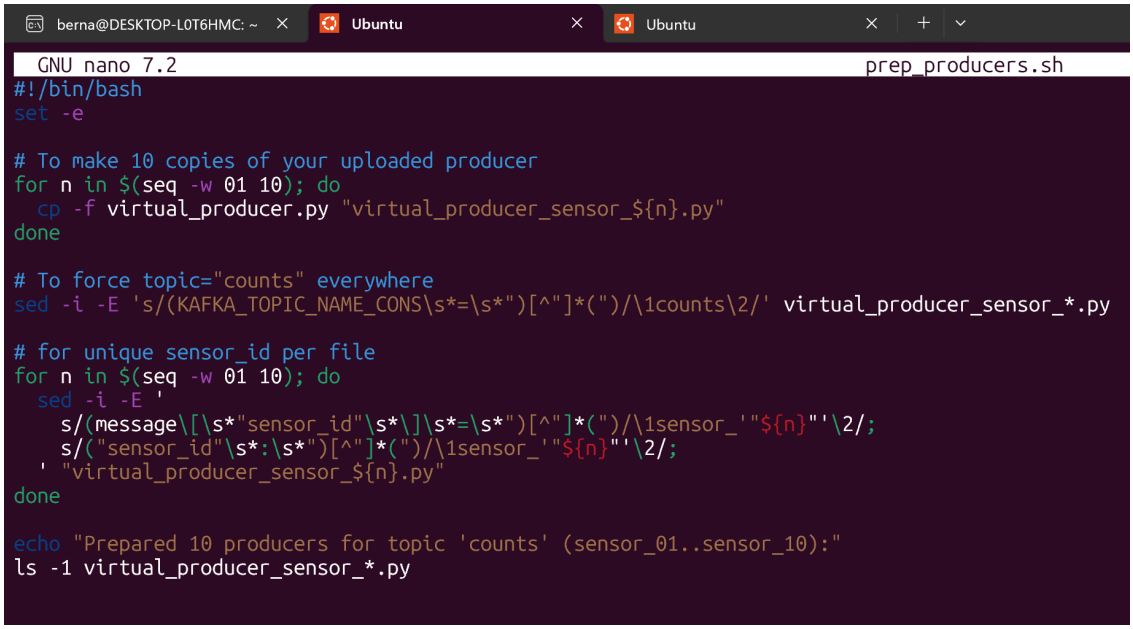
# To force topic="counts" everywhere
```

```
sed -i -E 's/(KAFKA_TOPIC_NAME_CONS\s*=\s*")[^"]*(")/\1counts\2/'
virtual_producer_sensor_*.py

# for unique sensor_id per file
for n in $(seq -w 01 10); do
  sed -i -E
"s/(message\[\s*\"sensor_id\"\s*\]\s*=\s*['\"]).*(['\"])/\1sensor_${n}\2/"
"virtual_producer_sensor_${n}.py"
  sed -i -E "s/(\"sensor_id\"\s*:\s*['\"]).*(['\"])/\1sensor_${n}\2/"
"virtual_producer_sensor_${n}.py"
done

echo "Prepared 10 producers for topic 'counts' (sensor_01..sensor_10):"
ls -1 virtual_producer_sensor_*.py
```

Just like this:



Once done, do not forget to save and input this command to make it executable:
`chmod +x prep_producers.sh`

```
berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ chmod +x prep_producers.sh
berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$
```

How the Script works

- #!/bin/bash designates Bash as the interpreter; set -e makes the script terminate at the first command failure to avoid partial/dirty changes.
- Loop 1 iterates base code of your producer to ten different files named virtual_producer_sensor_01.py … virtual_producer_sensor_10.py; seq -w 01 10 generates zero-padded numbers; cp-f will replace the files if they are already there.

- The sed -i -E operation changes only the quoted value of KAFKA_TOPIC_NAME_CONS with "counts" in all the files that have been copied by utilizing extended regex and in-place editing (GNU sed Manual, n.d.).
- Loop 2 runs in a file two sed commands to set a different sensor_id (sensor_01 … sensor_10) for both the message["sensor_id"] = "..." style and the dict/JSON style ("sensor_id": "..."), again by in-place, extended-regex replacements (GNU sed Manual, n.d.).
- Lastly, it outputs a brief confirmation and displays the filenames that have been prepared, one per line (ls -1)

Run created script for 10 copies

Now that we have the script, type ls to confirm that the script was created inside the project directory, and this also shows that there are no instances of the producer yet.

```
berna@DESKTOP-L0T6HMC: $ cd FinalProjDsci-2
berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ ls
prep_producers.sh  pyVenv  virtual_consumer.py  virtual_consumer.py:Zone.Identifier  virtual_producer.py  virtual_producer.py:Zone.Identifier
berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$
```

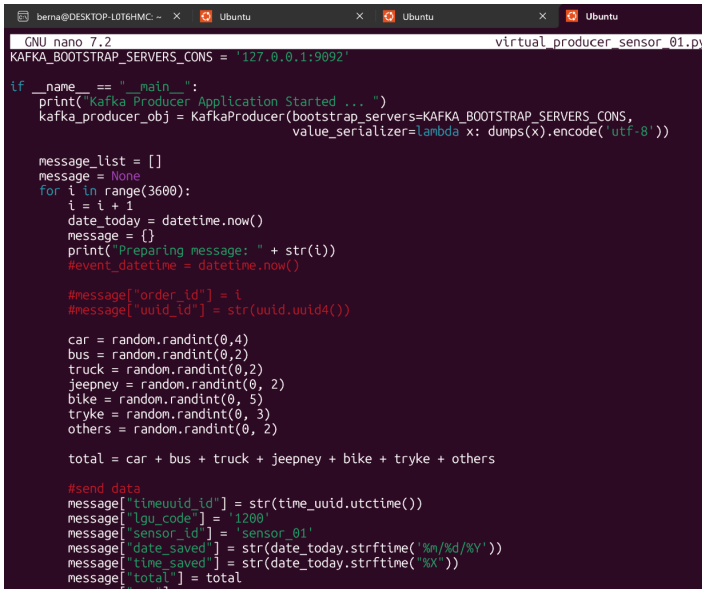
Run the script using the command `bash prep_producers.sh`:

```
berna@DESKTOP-L0T6HMC: $ cd FinalProjDsci-2
berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ ls
prep_producers.sh  pyVenv  virtual_consumer.py  virtual_consumer.py:Zone.Identifier  virtual_producer.py  virtual_producer.py:Zone.Identifier
berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ bash prep_producers.sh
Prepared 10 producers for topic 'counts' (sensor_01..sensor_10):
virtual_producer_sensor_01.py
virtual_producer_sensor_02.py
virtual_producer_sensor_03.py
virtual_producer_sensor_04.py
virtual_producer_sensor_05.py
virtual_producer_sensor_06.py
virtual_producer_sensor_07.py
virtual_producer_sensor_08.py
virtual_producer_sensor_09.py
virtual_producer_sensor_10.py
berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ ls
prep_producers.sh                  virtual_producer.py                virtual_producer_sensor_03.py  virtual_producer_sensor_07.py
pyVenv                             virtual_producer.py:Zone.Identifier  virtual_producer_sensor_04.py  virtual_producer_sensor_08.py
virtual_consumer.py                virtual_producer_sensor_01.py      virtual_producer_sensor_05.py  virtual_producer_sensor_09.py
virtual_consumer.py:Zone.Identifier  virtual_producer_sensor_02.py    virtual_producer_sensor_06.py  virtual_producer_sensor_10.py
berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$
```

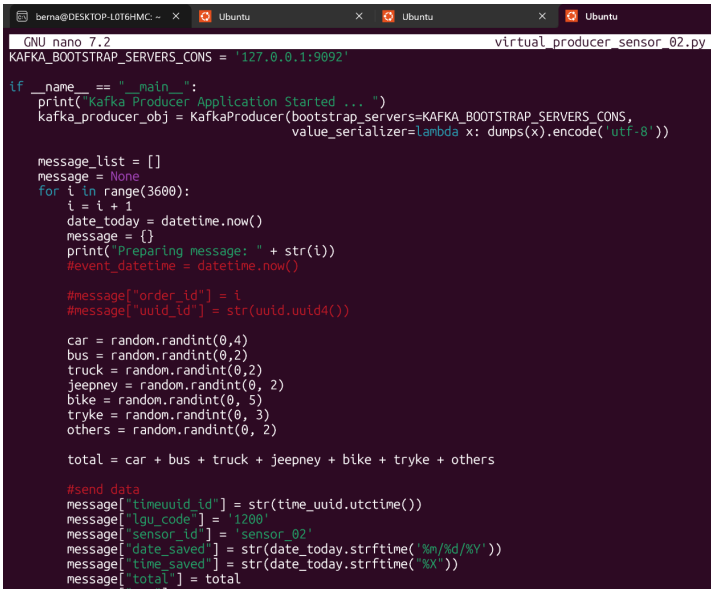
And inputting ls shows that there are now 10 instances of the virtual_producer.py. In order to confirm it, we will look inside each code and check if the sensor_ids are unique, that is named from sensor_01 to sensor_10. In order to do that we input the command: `nano virtual_producer_sensor_(input number).py`

Screenshots of Each producer codes

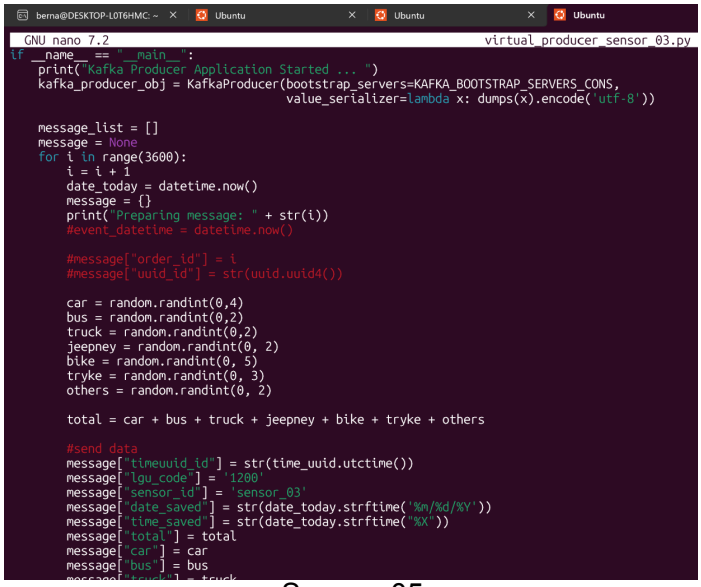
Sensor_01



Sensor_02



Sensor_03



```
GNU nano 7.2                                    virtual_producer_sensor_03.py
if __name__ == "__main__":
    print("Kafka Producer Application Started ... ")
    kafka_producer_obj = KafkaProducer(bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS_CONS,
                                       value_serializer=lambda x: dumps(x).encode('utf-8'))

    message_list = []
    message = None
    for i in range(3600):
        i = i + 1
        date_today = datetime.now()
        message = {}
        print("Preparing message: " + str(i))
        #event_datetime = datetime.now()

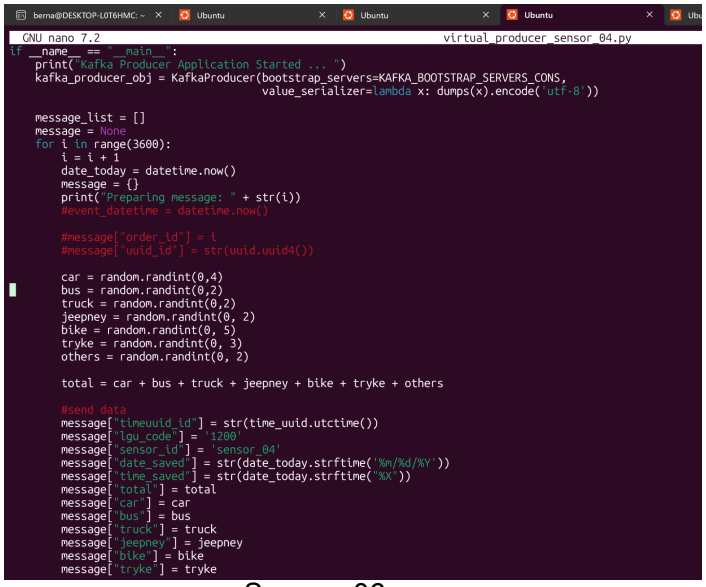
        #message["order_id"] = i
        #message["uuid_id"] = str(uuid.uuid4())

        car = random.randint(0,4)
        bus = random.randint(0,2)
        truck = random.randint(0,2)
        jeepney = random.randint(0, 2)
        bike = random.randint(0, 5)
        tryke = random.randint(0, 3)
        others = random.randint(0, 2)

        total = car + bus + truck + jeepney + bike + tryke + others

        #send_data
        message["timeuuid_id"] = str(time_uuid.utctime())
        message["lgu_code"] = '1200'
        message["sensor_id"] = 'sensor_03'
        message["date_saved"] = str(date_today.strftime('%m/%d/%Y'))
        message["time_saved"] = str(date_today.strftime("%X"))
        message["total"] = total
        message["car"] = car
        message["bus"] = bus
        message["truck"] = truck
```

Sensor_04



```
GNU nano 7.2                                    virtual_producer_sensor_04.py
if __name__ == "__main__":
    print("Kafka Producer Application Started ... ")
    kafka_producer_obj = KafkaProducer(bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS_CONS,
                                       value_serializer=lambda x: dumps(x).encode('utf-8'))

    message_list = []
    message = None
    for i in range(3600):
        i = i + 1
        date_today = datetime.now()
        message = {}
        print("Preparing message: " + str(i))
        #event_datetime = datetime.now()

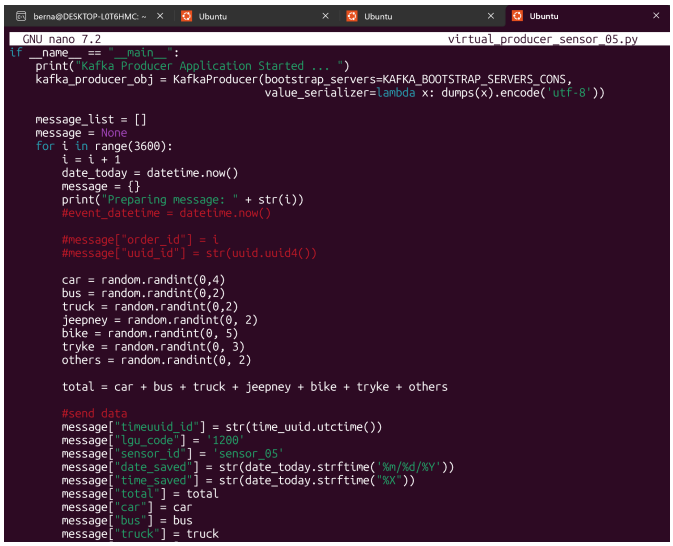
        #message["order_id"] = i
        #message["uuid_id"] = str(uuid.uuid4())

        car = random.randint(0,4)
        bus = random.randint(0,2)
        truck = random.randint(0,2)
        jeepney = random.randint(0, 2)
        bike = random.randint(0, 5)
        tryke = random.randint(0, 3)
        others = random.randint(0, 2)

        total = car + bus + truck + jeepney + bike + tryke + others

        #send_data
        message["timeuuid_id"] = str(time_uuid.utctime())
        message["lgu_code"] = '1200'
        message["sensor_id"] = 'sensor_04'
        message["date_saved"] = str(date_today.strftime('%m/%d/%Y'))
        message["time_saved"] = str(date_today.strftime("%X"))
        message["total"] = total
        message["car"] = car
        message["bus"] = bus
        message["truck"] = truck
        message["jeepney"] = jeepney
        message["bike"] = bike
        message["tryke"] = tryke
```

Sensor_05



```
GNU nano 7.2                                    virtual_producer_sensor_05.py
if __name__ == "__main__":
    print("Kafka Producer Application Started ... ")
    kafka_producer_obj = KafkaProducer(bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS_CONS,
                                       value_serializer=lambda x: dumps(x).encode('utf-8'))

    message_list = []
    message = None
    for i in range(3600):
        i = i + 1
        date_today = datetime.now()
        message = {}
        print("Preparing message: " + str(i))
        #event_datetime = datetime.now()

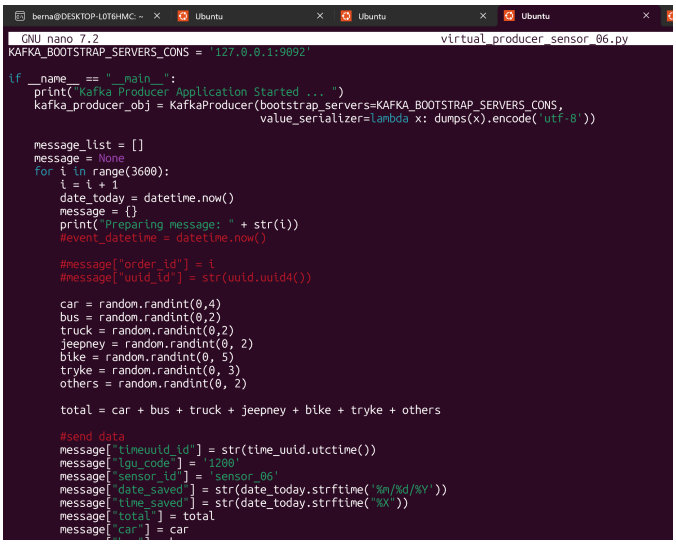
        #message["order_id"] = i
        #message["uuid_id"] = str(uuid.uuid4())

        car = random.randint(0,4)
        bus = random.randint(0,2)
        truck = random.randint(0,2)
        jeepney = random.randint(0, 2)
        bike = random.randint(0, 5)
        tryke = random.randint(0, 3)
        others = random.randint(0, 2)

        total = car + bus + truck + jeepney + bike + tryke + others

        #send_data
        message["timeuuid_id"] = str(time_uuid.utctime())
        message["lgu_code"] = '1200'
        message["sensor_id"] = 'sensor_05'
        message["date_saved"] = str(date_today.strftime('%m/%d/%Y'))
        message["time_saved"] = str(date_today.strftime("%X"))
        message["total"] = total
        message["car"] = car
        message["bus"] = bus
        message["truck"] = truck
```

Sensor_06



```
GNU nano 7.2                                    virtual_producer_sensor_06.py
KAFKA_BOOTSTRAP_SERVERS_CONS = '127.0.0.1:9092'

if __name__ == "__main__":
    print("Kafka Producer Application Started ... ")
    kafka_producer_obj = KafkaProducer(bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS_CONS,
                                       value_serializer=lambda x: dumps(x).encode('utf-8'))

    message_list = []
    message = None
    for i in range(3600):
        i = i + 1
        date_today = datetime.now()
        message = {}
        print("Preparing message: " + str(i))
        #event_datetime = datetime.now()

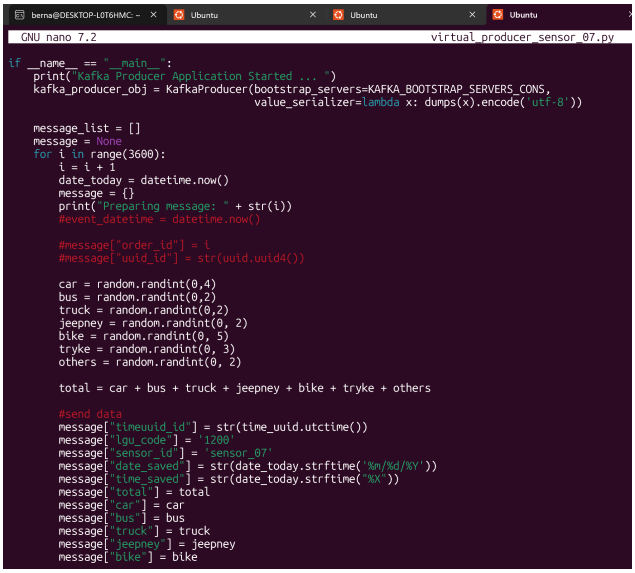
        #message["order_id"] = i
        #message["uuid_id"] = str(uuid.uuid4())

        car = random.randint(0,4)
        bus = random.randint(0,2)
        truck = random.randint(0,2)
        jeepney = random.randint(0, 2)
        bike = random.randint(0, 5)
        tryke = random.randint(0, 3)
        others = random.randint(0, 2)

        total = car + bus + truck + jeepney + bike + tryke + others

        #send_data
        message["timeuuid_id"] = str(time_uuid.utctime())
        message["lgu_code"] = '1200'
        message["sensor_id"] = 'sensor_06'
        message["date_saved"] = str(date_today.strftime('%m/%d/%Y'))
        message["time_saved"] = str(date_today.strftime("%X"))
        message["total"] = total
        message["car"] = car
```

Sensor_07



```
GNU nano 7.2                                    virtual_producer_sensor_07.py
if __name__ == "__main__":
    print("Kafka Producer Application Started ... ")
    kafka_producer_obj = KafkaProducer(bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS_CONS,
                                       value_serializer=lambda x: dumps(x).encode('utf-8'))

    message_list = []
    message = None
    for i in range(3600):
        i = i + 1
        date_today = datetime.now()
        message = {}
        print("Preparing message: " + str(i))
        #event_datetime = datetime.now()

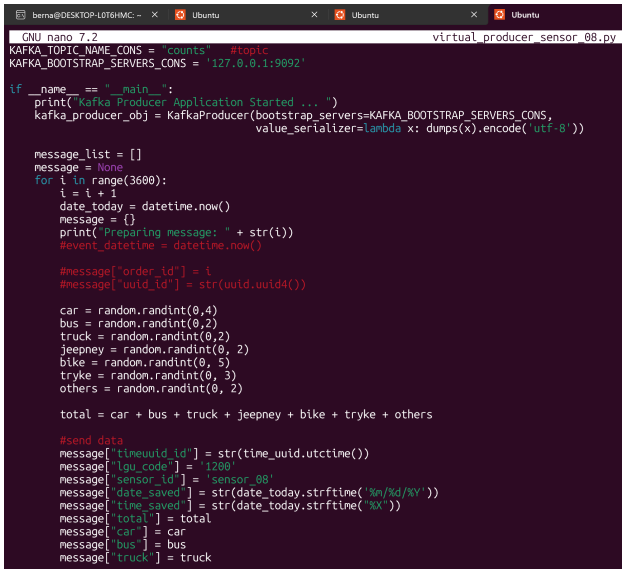
        #message["order_id"] = i
        #message["uuid_id"] = str(uuid.uuid4())

        car = random.randint(0,4)
        bus = random.randint(0,2)
        truck = random.randint(0,2)
        jeepney = random.randint(0, 2)
        bike = random.randint(0, 5)
        tryke = random.randint(0, 3)
        others = random.randint(0, 2)

        total = car + bus + truck + jeepney + bike + tryke + others

        #send_data
        message["timeuuid_id"] = str(time_uuid.utctime())
        message["lgu_code"] = '1200'
        message["sensor_id"] = 'sensor_07'
        message["date_saved"] = str(date_today.strftime('%m/%d/%Y'))
        message["time_saved"] = str(date_today.strftime("%X"))
        message["total"] = total
        message["car"] = car
        message["bus"] = bus
        message["truck"] = truck
        message["jeepney"] = jeepney
        message["bike"] = bike
```

Sensor_08



```
GNU nano 7.2                                    virtual_producer_sensor_08.py
KAFKA_TOPIC_NAME_CONS = "counts"   #topic
KAFKA_BOOTSTRAP_SERVERS_CONS = '127.0.0.1:9092'

if __name__ == "__main__":
    print("Kafka Producer Application Started ... ")
    kafka_producer_obj = KafkaProducer(bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS_CONS,
                                       value_serializer=lambda x: dumps(x).encode('utf-8'))

    message_list = []
    message = None
    for i in range(3600):
        i = i + 1
        date_today = datetime.now()
        message = {}
        print("Preparing message: " + str(i))
        #event_datetime = datetime.now()

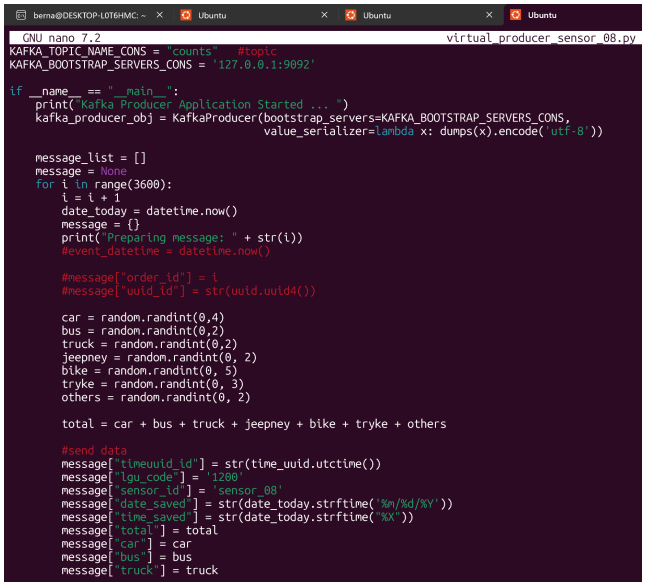
        #message["order_id"] = i
        #message["uuid_id"] = str(uuid.uuid4())

        car = random.randint(0,4)
        bus = random.randint(0,2)
        truck = random.randint(0,2)
        jeepney = random.randint(0, 2)
        bike = random.randint(0, 5)
        tryke = random.randint(0, 3)
        others = random.randint(0, 2)

        total = car + bus + truck + jeepney + bike + tryke + others

        #send_data
        message["timeuuid_id"] = str(time_uuid.utctime())
        message["lgu_code"] = '1200'
        message["sensor_id"] = 'sensor_08'
        message["date_saved"] = str(date_today.strftime('%m/%d/%Y'))
        message["time_saved"] = str(date_today.strftime("%X"))
        message["total"] = total
        message["car"] = car
        message["bus"] = bus
        message["truck"] = truck
```

Sensor_09



```
  GNU nano 7.2                                  virtual_producer_sensor_08.py
KAFKA_TOPIC_NAME_CONS = "counts"   #topic
KAFKA_BOOTSTRAP_SERVERS_CONS = '127.0.0.1:9092'

if __name__ == "__main__":
    print("Kafka Producer Application Started ... ")
    kafka_producer_obj = KafkaProducer(bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS_CONS,
                                       value_serializer=lambda x: dumps(x).encode('utf-8'))

    message_list = []
    message = None
    for i in range(3600):
        i = i + 1
        date_today = datetime.now()
        message = {}
        print("Preparing message: " + str(i))
        #event_datetime = datetime.now()

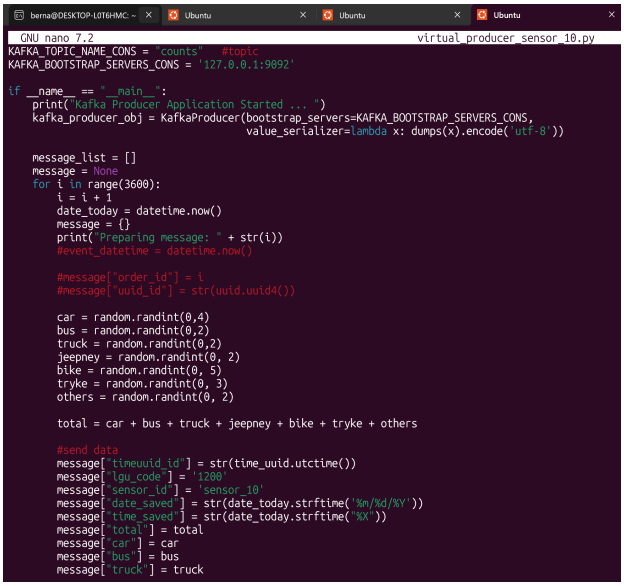
        #message["order_id"] = i
        #message["uuid_id"] = str(uuid.uuid4())

        car = random.randint(0,4)
        bus = random.randint(0,2)
        truck = random.randint(0,2)
        jeepney = random.randint(0, 2)
        bike = random.randint(0, 5)
        tryke = random.randint(0, 3)
        others = random.randint(0, 2)

        total = car + bus + truck + jeepney + bike + tryke + others

        #send data
        message["timeuuid_id"] = str(time_uuid.utctime())
        message["lgu_code"] = '1209'
        message["sensor_id"] = 'sensor_08'
        message["date_saved"] = str(date_today.strftime('%m/%d/%Y'))
        message["time_saved"] = str(date_today.strftime("%X"))
        message["total"] = total
        message["car"] = car
        message["bus"] = bus
        message["truck"] = truck
```

Sensor_10



```
  GNU nano 7.2                                  virtual_producer_sensor_10.py
KAFKA_TOPIC_NAME_CONS = "counts"   #topic
KAFKA_BOOTSTRAP_SERVERS_CONS = '127.0.0.1:9092'

if __name__ == "__main__":
    print("Kafka Producer Application Started ... ")
    kafka_producer_obj = KafkaProducer(bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS_CONS,
                                       value_serializer=lambda x: dumps(x).encode('utf-8'))

    message_list = []
    message = None
    for i in range(3600):
        i = i + 1
        date_today = datetime.now()
        message = {}
        print("Preparing message: " + str(i))
        #event_datetime = datetime.now()

        #message["order_id"] = i
        #message["uuid_id"] = str(uuid.uuid4())

        car = random.randint(0,4)
        bus = random.randint(0,2)
        truck = random.randint(0,2)
        jeepney = random.randint(0, 2)
        bike = random.randint(0, 5)
        tryke = random.randint(0, 3)
        others = random.randint(0, 2)

        total = car + bus + truck + jeepney + bike + tryke + others

        #send data
        message["timeuuid_id"] = str(time_uuid.utctime())
        message["lgu_code"] = '1209'
        message["sensor_id"] = 'sensor_10'
        message["date_saved"] = str(date_today.strftime('%m/%d/%Y'))
        message["time_saved"] = str(date_today.strftime("%X"))
        message["total"] = total
        message["car"] = car
        message["bus"] = bus
        message["truck"] = truck
```

As you can see in the screenshots, the sensor_id is changed to its respective sensor number, which is from sensor_01 to sensor_10.

Lastly here is a final Check to see the 10 sensors corresponding sensor_id, using `grep -Hn 'sensor_id' virtual_producer_sensor_*.py`:

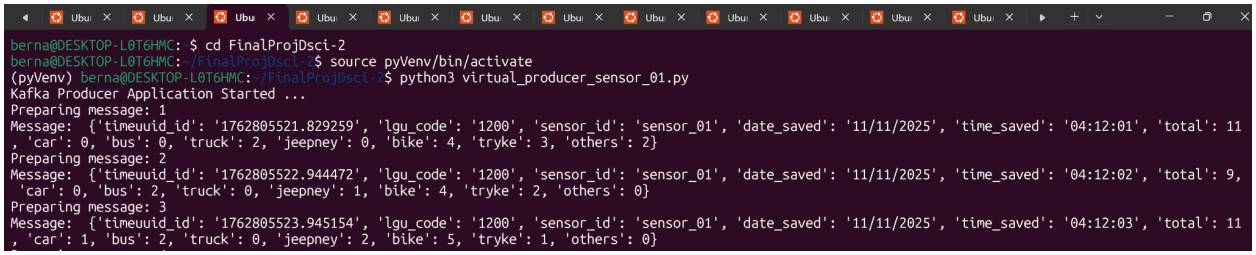
```
berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ ls
prep_producers.sh                   virtual_producer.py                virtual_producer_s
pyVenv                              virtual_producer.py:Zone.Identifier  virtual_producer_s
virtual_consumer.py                 virtual_producer_sensor_01.py      virtual_producer_s
virtual_consumer.py:Zone.Identifier  virtual_producer_sensor_02.py      virtual_producer_s
berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ nano virtual_producer_sensor_01.py
berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ nano virtual_producer_sensor_02.py
berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ nano virtual_producer_sensor_03.py
berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ nano virtual_producer_sensor_04.py
berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ nano virtual_producer_sensor_05.py
berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ nano virtual_producer_sensor_06.py
berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ nano virtual_producer_sensor_07.py
berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ nano virtual_producer_sensor_08.py
berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ nano virtual_producer_sensor_09.py
berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ nano virtual_producer_sensor_10.py
berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ grep -Hn 'sensor_id' virtual_producer_sensor_*.py
virtual_producer_sensor_01.py:45:        message["sensor_id"] = 'sensor_01'
virtual_producer_sensor_02.py:45:        message["sensor_id"] = 'sensor_02'
virtual_producer_sensor_03.py:45:        message["sensor_id"] = 'sensor_03'
virtual_producer_sensor_04.py:45:        message["sensor_id"] = 'sensor_04'
virtual_producer_sensor_05.py:45:        message["sensor_id"] = 'sensor_05'
virtual_producer_sensor_06.py:45:        message["sensor_id"] = 'sensor_06'
virtual_producer_sensor_07.py:45:        message["sensor_id"] = 'sensor_07'
virtual_producer_sensor_08.py:45:        message["sensor_id"] = 'sensor_08'
virtual_producer_sensor_09.py:45:        message["sensor_id"] = 'sensor_09'
virtual_producer_sensor_10.py:45:        message["sensor_id"] = 'sensor_10'
```

Run all 10 Producers

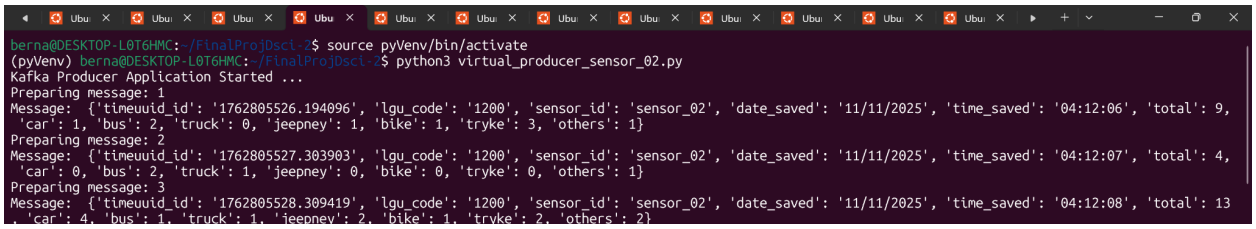
Before running the producers, we are inside a python environment to isolate package versions (kafka-python, cassandra-driver, etc.) and the interpreter, so tje 10 processes all run against the same dependencies. Also, if you have not yet installed this package: `python3 -m pip install kafka-python time-uuid`, to run the 10 producers:

```
(pyVenv) berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ python3 -m pip install kafka-python time-uuid
Collecting kafka-python
  Downloading kafka_python-2.2.15-py2.py3-none-any.whl.metadata (10.0 kB)
Collecting time-uuid
  Downloading time-uuid-0.2.0.tar.gz (3.6 kB)
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Downloading kafka_python-2.2.15-py2.py3-none-any.whl (309 kB)
Building wheels for collected packages: time-uuid
  Building wheel for time-uuid (pyproject.toml) ... done
  Created wheel for time-uuid: filename=time_uuid-0.2.0-py3-none-any.whl size=3846 sha256=9baa0b2f5e9914c2cceef0861a04b8eed6cffa79dc6776c16e1a042626d494ed
  Stored in directory: /home/berna/.cache/pip/wheels/69/d2/95/1f6c044fda1442e9871ad585f288c22b33b387d8ddc6d958b3
Successfully built time-uuid
Installing collected packages: time-uuid, kafka-python
Successfully installed kafka-python-2.2.15 time-uuid-0.2.0
(pyVenv) berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$
```

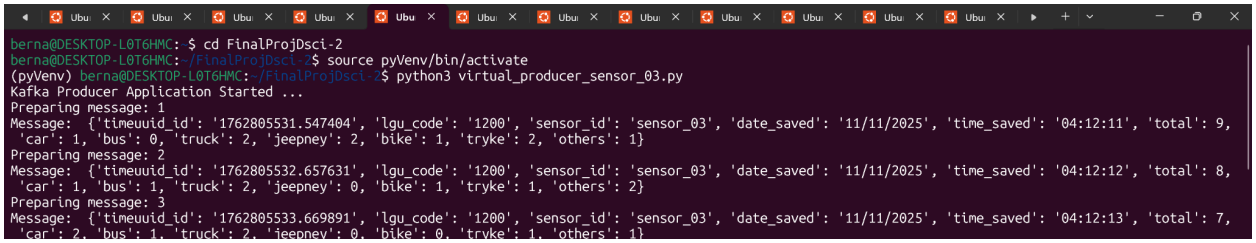
Now inside the environment, run all the producers simultaneously so that the data produced will be sent to a topic in Kafka. Below are the images of the 10 producers running simultaneously:



```
berna@DESKTOP-L0T6HMC: $ cd FinalProjDsci-2
berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ source pyVenv/bin/activate
(pyVenv) berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ python3 virtual_producer_sensor_01.py
Kafka Producer Application Started ...
Preparing message: 1
Message:  {'timeuuid_id': '1762805521.829259', 'lgu_code': '1200', 'sensor_id': 'sensor_01', 'date_saved': '11/11/2025', 'time_saved': '04:12:01', 'total': 11
, 'car': 0, 'bus': 0, 'truck': 2, 'jeepney': 0, 'bike': 4, 'tryke': 3, 'others': 2}
Preparing message: 2
Message:  {'timeuuid_id': '1762805522.944472', 'lgu_code': '1200', 'sensor_id': 'sensor_01', 'date_saved': '11/11/2025', 'time_saved': '04:12:02', 'total': 9,
 'car': 0, 'bus': 2, 'truck': 0, 'jeepney': 1, 'bike': 4, 'tryke': 2, 'others': 0}
Preparing message: 3
Message:  {'timeuuid_id': '1762805523.945154', 'lgu_code': '1200', 'sensor_id': 'sensor_01', 'date_saved': '11/11/2025', 'time_saved': '04:12:03', 'total': 11
, 'car': 1, 'bus': 2, 'truck': 0, 'jeepney': 2, 'bike': 5, 'tryke': 1, 'others': 0}
```



```
berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ source pyVenv/bin/activate
(pyVenv) berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ python3 virtual_producer_sensor_02.py
Kafka Producer Application Started ...
Preparing message: 1
Message:  {'timeuuid_id': '1762805526.194096', 'lgu_code': '1200', 'sensor_id': 'sensor_02', 'date_saved': '11/11/2025', 'time_saved': '04:12:06', 'total': 9,
 'car': 1, 'bus': 2, 'truck': 0, 'jeepney': 1, 'bike': 1, 'tryke': 3, 'others': 1}
Preparing message: 2
Message:  {'timeuuid_id': '1762805527.303903', 'lgu_code': '1200', 'sensor_id': 'sensor_02', 'date_saved': '11/11/2025', 'time_saved': '04:12:07', 'total': 4,
 'car': 0, 'bus': 2, 'truck': 1, 'jeepney': 0, 'bike': 0, 'tryke': 0, 'others': 1}
Preparing message: 3
Message:  {'timeuuid_id': '1762805528.309419', 'lgu_code': '1200', 'sensor_id': 'sensor_02', 'date_saved': '11/11/2025', 'time_saved': '04:12:08', 'total': 13
, 'car': 4, 'bus': 1, 'truck': 1, 'jeepney': 2, 'bike': 1, 'tryke': 2, 'others': 2}
```



```
berna@DESKTOP-L0T6HMC: $ cd FinalProjDsci-2
berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ source pyVenv/bin/activate
(pyVenv) berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ python3 virtual_producer_sensor_03.py
Kafka Producer Application Started ...
Preparing message: 1
Message:  {'timeuuid_id': '1762805531.547404', 'lgu_code': '1200', 'sensor_id': 'sensor_03', 'date_saved': '11/11/2025', 'time_saved': '04:12:11', 'total': 9,
 'car': 1, 'bus': 0, 'truck': 2, 'jeepney': 2, 'bike': 1, 'tryke': 2, 'others': 1}
Preparing message: 2
Message:  {'timeuuid_id': '1762805532.657631', 'lgu_code': '1200', 'sensor_id': 'sensor_03', 'date_saved': '11/11/2025', 'time_saved': '04:12:12', 'total': 8,
 'car': 1, 'bus': 1, 'truck': 2, 'jeepney': 0, 'bike': 1, 'tryke': 1, 'others': 2}
Preparing message: 3
Message:  {'timeuuid_id': '1762805533.669891', 'lgu_code': '1200', 'sensor_id': 'sensor_03', 'date_saved': '11/11/2025', 'time_saved': '04:12:13', 'total': 7,
 'car': 2, 'bus': 1, 'truck': 2, 'jeepney': 0, 'bike': 0, 'tryke': 1, 'others': 1}
```



```
berna@DESKTOP-L0T6HMC: $ cd FinalProjDsci-2
berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ source pyVenv/bin/activate
(pyVenv) berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ python3 virtual_producer_sensor_04.py
Kafka Producer Application Started ...
Preparing message: 1
Message:  {'timeuuid_id': '1762805535.304762', 'lgu_code': '1200', 'sensor_id': 'sensor_04', 'date_saved': '11/11/2025', 'time_saved': '04:12:15', 'total': 9,
 'car': 1, 'bus': 0, 'truck': 1, 'jeepney': 1, 'bike': 4, 'tryke': 2, 'others': 0}
Preparing message: 2
Message:  {'timeuuid_id': '1762805536.417879', 'lgu_code': '1200', 'sensor_id': 'sensor_04', 'date_saved': '11/11/2025', 'time_saved': '04:12:16', 'total': 8,
 'car': 3, 'bus': 1, 'truck': 2, 'jeepney': 0, 'bike': 2, 'tryke': 0, 'others': 0}
Preparing message: 3
Message:  {'timeuuid_id': '1762805537.418968', 'lgu_code': '1200', 'sensor_id': 'sensor_04', 'date_saved': '11/11/2025', 'time_saved': '04:12:17', 'total': 7,
 'car': 2, 'bus': 0, 'truck': 1, 'jeepney': 1, 'bike': 1, 'tryke': 2, 'others': 0}
```

```
berna@DESKTOP-L0T6HMC: $ cd FinalProjDsci-2
berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ source pyVenv/bin/activate
(pyVenv) berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ python3 virtual_producer_sensor_06.py
Kafka Producer Application Started ...
Preparing message: 1
Message:  {'timeuuid_id': '1762805544.199667', 'lgu_code': '1200', 'sensor_id': 'sensor_06', 'date_saved': '11/11/2025', 'time_saved': '04:12:24', 'total': 11
, 'car': 3, 'bus': 1, 'truck': 0, 'jeepney': 1, 'bike': 2, 'tryke': 2, 'others': 2}
Preparing message: 2
Message:  {'timeuuid_id': '1762805545.306895', 'lgu_code': '1200', 'sensor_id': 'sensor_06', 'date_saved': '11/11/2025', 'time_saved': '04:12:25', 'total': 15
, 'car': 4, 'bus': 2, 'truck': 2, 'jeepney': 2, 'bike': 3, 'tryke': 1, 'others': 1}
Preparing message: 3
Message:  {'timeuuid_id': '1762805546.307932', 'lgu_code': '1200', 'sensor_id': 'sensor_06', 'date_saved': '11/11/2025', 'time_saved': '04:12:26', 'total': 15
, 'car': 3, 'bus': 2, 'truck': 0, 'jeepney': 2, 'bike': 5, 'tryke': 3, 'others': 0}
```

```
berna@DESKTOP-L0T6HMC: $ cd FinalProjDsci-2
berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ source pyVenv/bin/activate
(pyVenv) berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ python3 virtual_producer_sensor_07.py
Kafka Producer Application Started ...
Preparing message: 1
Message:  {'timeuuid_id': '1762805549.003716', 'lgu_code': '1200', 'sensor_id': 'sensor_07', 'date_saved': '11/11/2025', 'time_saved': '04:12:29', 'total': 11
, 'car': 3, 'bus': 0, 'truck': 2, 'jeepney': 2, 'bike': 0, 'tryke': 2, 'others': 2}
Preparing message: 2
Message:  {'timeuuid_id': '1762805550.112573', 'lgu_code': '1200', 'sensor_id': 'sensor_07', 'date_saved': '11/11/2025', 'time_saved': '04:12:30', 'total': 10
, 'car': 1, 'bus': 2, 'truck': 1, 'jeepney': 2, 'bike': 2, 'tryke': 2, 'others': 0}
Preparing message: 3
Message:  {'timeuuid_id': '1762805551.113261', 'lgu_code': '1200', 'sensor_id': 'sensor_07', 'date_saved': '11/11/2025', 'time_saved': '04:12:31', 'total': 9,
 'car': 0, 'bus': 1, 'truck': 2, 'jeepney': 0, 'bike': 3, 'tryke': 3, 'others': 0}
```

```
berna@DESKTOP-L0T6HMC: $ cd FinalProjDsci-2
berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ source pyVenv/bin/activate
(pyVenv) berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ python3 virtual_producer_sensor_08.py
Kafka Producer Application Started ...
Preparing message: 1
Message:  {'timeuuid_id': '1762805559.404098', 'lgu_code': '1200', 'sensor_id': 'sensor_08', 'date_saved': '11/11/2025', 'time_saved': '04:12:39', 'total': 12
, 'car': 3, 'bus': 2, 'truck': 0, 'jeepney': 1, 'bike': 4, 'tryke': 2, 'others': 0}
Preparing message: 2
Message:  {'timeuuid_id': '1762805560.548403', 'lgu_code': '1200', 'sensor_id': 'sensor_08', 'date_saved': '11/11/2025', 'time_saved': '04:12:40', 'total': 12
, 'car': 3, 'bus': 1, 'truck': 0, 'jeepney': 1, 'bike': 3, 'tryke': 2, 'others': 2}
Preparing message: 3
Message:  {'timeuuid_id': '1762805561.559575', 'lgu_code': '1200', 'sensor_id': 'sensor_08', 'date_saved': '11/11/2025', 'time_saved': '04:12:41', 'total': 9,
 'car': 2, 'bus': 1, 'truck': 1, 'jeepney': 1, 'bike': 3, 'tryke': 1, 'others': 0}
```

```
berna@DESKTOP-L0T6HMC: $ cd FinalProjDsci-2
berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ source pyVenv/bin/activate
(pyVenv) berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ python3 virtual_producer_sensor_09.py
Kafka Producer Application Started ...
Preparing message: 1
Message:  {'timeuuid_id': '1762805557.379781', 'lgu_code': '1200', 'sensor_id': 'sensor_09', 'date_saved': '11/11/2025', 'time_saved': '04:12:37', 'total': 3,
 'car': 0, 'bus': 0, 'truck': 1, 'jeepney': 1, 'bike': 0, 'tryke': 1, 'others': 0}
Preparing message: 2
Message:  {'timeuuid_id': '1762805558.48736', 'lgu_code': '1200', 'sensor_id': 'sensor_09', 'date_saved': '11/11/2025', 'time_saved': '04:12:38', 'total': 11,
 'car': 1, 'bus': 2, 'truck': 2, 'jeepney': 1, 'bike': 2, 'tryke': 1, 'others': 2}
Preparing message: 3
Message:  {'timeuuid_id': '1762805559.489109', 'lgu_code': '1200', 'sensor_id': 'sensor_09', 'date_saved': '11/11/2025', 'time_saved': '04:12:39', 'total': 9,
 'car': 1, 'bus': 2, 'truck': 0, 'jeepney': 0, 'bike': 4, 'tryke': 2, 'others': 0}
```

```
berna@DESKTOP-L0T6HMC:~$ cd FinalProjDsci-2
berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ source pyVenv/bin/activate
(pyVenv) berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ python3 virtual_producer_sensor_10.py
Kafka Producer Application Started ...
Preparing message: 1
Message:  {'timeuuid_id': '1762805564.652213', 'lgu_code': '1200', 'sensor_id': 'sensor_10', 'date_saved': '11/11/2025', 'time_saved': '04:12:44', 'total': 10
, 'car': 1, 'bus': 0, 'truck': 2, 'jeepney': 2, 'bike': 0, 'tryke': 3, 'others': 2}
Preparing message: 2
Message:  {'timeuuid_id': '1762805565.760312', 'lgu_code': '1200', 'sensor_id': 'sensor_10', 'date_saved': '11/11/2025', 'time_saved': '04:12:45', 'total': 9,
 'car': 1, 'bus': 0, 'truck': 2, 'jeepney': 0, 'bike': 5, 'tryke': 0, 'others': 1}
Preparing message: 3
Message:  {'timeuuid_id': '1762805566.76196', 'lgu_code': '1200', 'sensor_id': 'sensor_10', 'date_saved': '11/11/2025', 'time_saved': '04:12:46', 'total': 7,
'car': 0, 'bus': 1, 'truck': 1, 'jeepney': 1, 'bike': 0, 'tryke': 2, 'others': 2}
```

Now we proceed in checking if Kafka is properly working by running the consumer. The consumer should display the data sent by the different producers, with the code above that was given for virtual_consumer.py this should be the output:

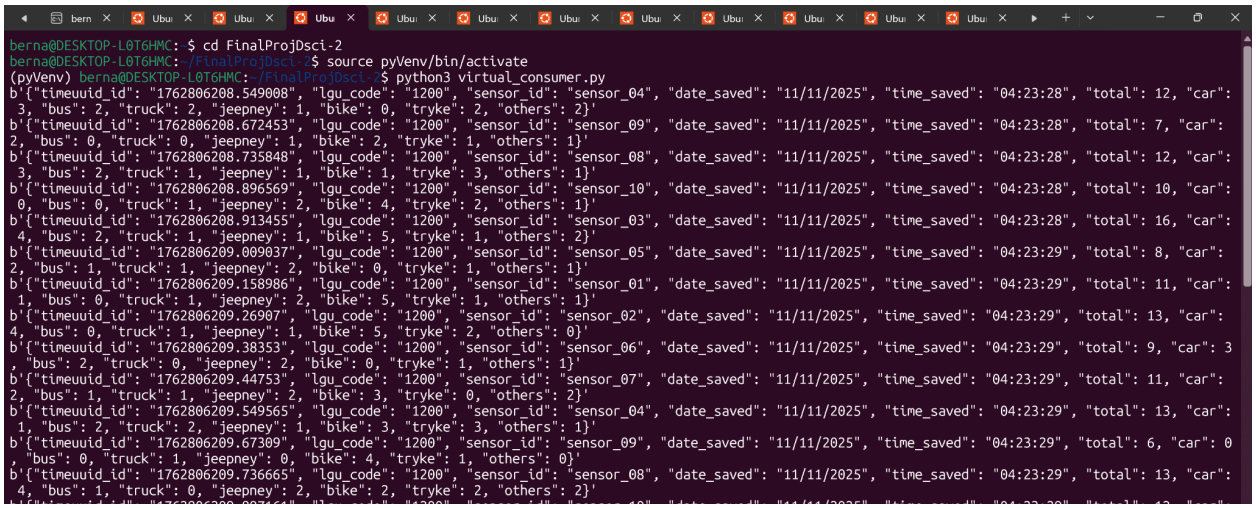
```
 GNU nano 7.2
# -*- coding: utf-8 -*-

from kafka import KafkaConsumer

consumer = KafkaConsumer(
    'counts',
    bootstrap_servers = ['127.0.0.1:9092'],
    auto_offset_reset = 'latest',
    enable_auto_commit = True
    )

for message in consumer:
    message = message.value
    print(message)
```

Again inside the same python environment, run the consumer using python3 virtual_consumer.py:

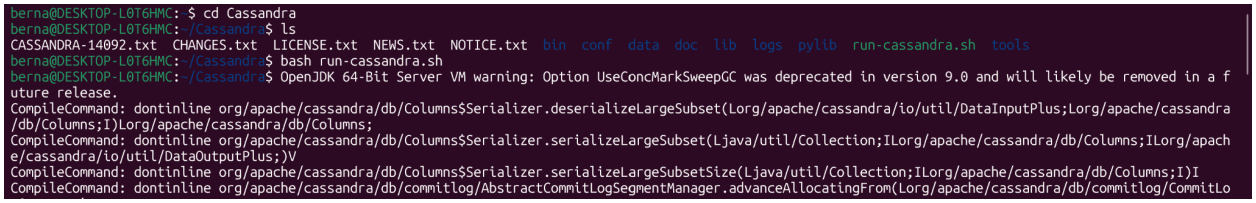


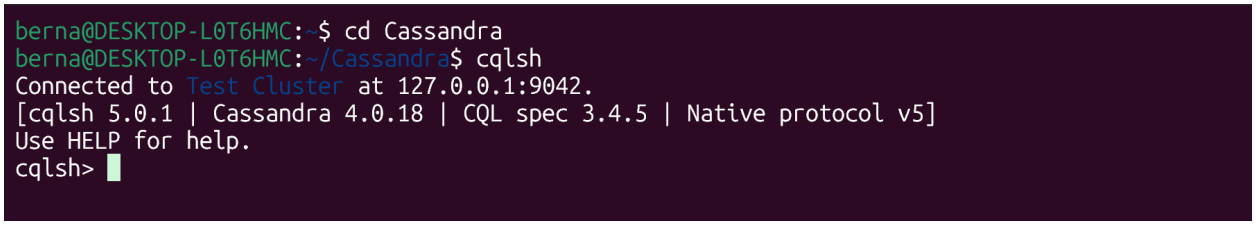
The output above shows that Kafka is working since both Producer and Consumer are communicating, that being said we now proceed to Apache Cassandra.

Create Keyspace

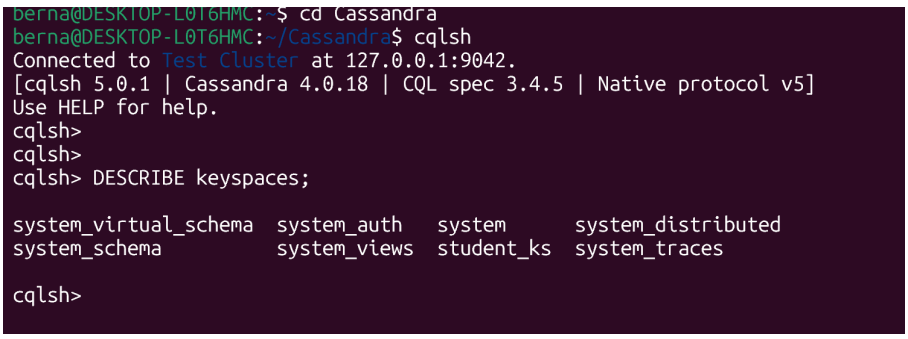
To start, we must first start and run Cassandra, and after that we proceed to creating the Keyspace which is the database for this project (Apache Cassandra, n.d.; DataStax, n.d.-a).

Run Cassandra





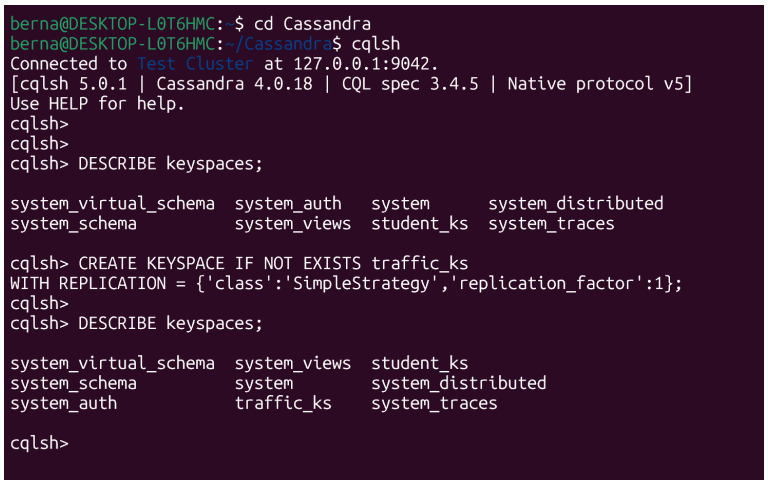
Current Keyspaces using `DESCRIBE keyspaces;`



Now we proceed to create the keyspace using this command:

```
CREATE KEYSPACE traffic_ks
WITH REPLICATION = {'class':'SimpleStrategy','replication_factor':1};
```

After that using `DESCRIBE keyspaces;` again, we now se that the new `traffic_ks` is now created.



Create Table

Using the keyspace created, we will input `USE traffic_ks;` so that the table will be inside that keyspace. Inside the cqlsh, input this code to create the table.

```
CREATE TABLE IF NOT EXISTS traffic_counts (
  timeuuid_id  timeuuid,
  lgu_code     text,
  sensor_id    text,
  date_saved   text,
  time_saved   text,
  car          int,
  total        int,
  bus          int,
  truck        int,
  jeepney      int,
  bike         int,
  tryke        int,
  others       int,
  PRIMARY KEY ((sensor_id), timeuuid_id)
) WITH CLUSTERING ORDER BY (timeuuid_id DESC);
```

Just like in the image below:

```
cqlsh> USE traffic_ks;
cqlsh:traffic_ks>
cqlsh:traffic_ks>
cqlsh:traffic_ks> CREATE TABLE IF NOT EXISTS traffic_counts (
             ... timeuuid_id timeuuid,
             ... lgu_code    text,
             ... sensor_id   text,
             ... date_saved  text,
             ... time_saved  text,
             ... total       int,
             ... car int,
             ... bus int,
             ... truck int,
             ... jeepney int,
             ... bike int,
             ... tryke int,
             ... others int,
             ... PRIMARY KEY ((sensor_id), timeuuid_id)
             ... ) WITH CLUSTERING ORDER BY (timeuuid_id DESC);
cqlsh:traffic_ks>
cqlsh:traffic_ks>
```

Lastly to verify that the table was successfully created, we input these codes: `DESCRIBE TABLE traffic_counts;` and `SELECT * FROM traffic_counts;` (Apache Cassandra, n.d.; DataStax, n.d.-b).

```
cqlsh:traffic_ks>
cqlsh:traffic_ks> DESCRIBE TABLE traffic_counts;

CREATE TABLE traffic_ks.traffic_counts (
    sensor_id text,
    timeuuid_id timeuuid,
    bike int,
    bus int,
    car int,
    date_saved text,
    jeepney int,
    lgu_code text,
    others int,
    time_saved text,
    total int,
    truck int,
    tryke int,
    PRIMARY KEY (sensor_id, timeuuid_id)
) WITH CLUSTERING ORDER BY (timeuuid_id DESC)
    AND additional_write_policy = '99p'
    AND bloom_filter_fp_chance = 0.01
    AND caching = {'keys': 'ALL', 'rows_per_partition': 'NONE'}
    AND comment = ''
    AND compaction = {'class': 'org.apache.cassandra.db.compaction.SizeTieredCompactionStrategy', 'max_threshold': '32', 'min_threshold': '4'}
    AND compression = {'chunk_length_in_kb': '16', 'class': 'org.apache.cassandra.io.compress.LZ4Compressor'}
    AND crc_check_chance = 1.0
    AND default_time_to_live = 0
    AND gc_grace_seconds = 864000
    AND max_index_interval = 2048
    AND memtable_flush_period_in_ms = 0
    AND min_index_interval = 128
    AND read_repair = 'BLOCKING'
    AND speculative_retry = '99p';

cqlsh:traffic_ks> SELECT * FROM traffic_counts;

 sensor_id | timeuuid_id | bike | bus | car | date_saved | jeepney | lgu_code | others | time_saved | total | truck | tryke
-----------+-------------+------+-----+-----+------------+---------+----------+--------+------------+-------+-------+-------

(0 rows)
cqlsh:traffic_ks>
```

By inputting those codes we see above that we have successfully created the table in Cassandra, now we proceed in the streaming of the data itself in Part 2.

Part 2

Now that we have already set up the producers, connectors, and Cassandra; and also verified that Kafka is streaming data, and the Cassandara table exists. We now, then proceed to create a python code that etches the latest batch of data that arrived in Kafka and then sends it to Cassandra so that the tables will be filled. Upon creating the python code, we use as reference the video of DataMaking (2019), in which we spin up a SparkSession with the Kafka and Cassandra connectors on the classpath, and then define a strict JSON schema mirroring the producer payload, and use readStream from Kafka. Below is the code to be used for the fetch_stream_to_cassandra.py that fetches the latest batch of data that arrived in Kafka and then sends (saves) this batch of data to Cassandra. Also, the fetch_stream_to_cassandra.py was created inside a new directory, cd ~/kafka-spark-cass, to group the Spark streaming code separate from producers, and to avoid path mistakes during spark-submit.

Code of fetch_stream_to_cassandra.py

```
from pyspark.sql import SparkSession, functions as F, types as T
from uuid import uuid1

schema = T.StructType([
    T.StructField("timeuuid_id", T.StringType()),
    T.StructField("lgu_code",    T.StringType()),
    T.StructField("sensor_id",   T.StringType()),
    T.StructField("date_saved",  T.StringType()),
    T.StructField("time_saved",  T.StringType()),
    T.StructField("total",       T.IntegerType()),
    T.StructField("car",         T.IntegerType()),
    T.StructField("bus",         T.IntegerType()),
    T.StructField("truck",       T.IntegerType()),
    T.StructField("jeepney",     T.IntegerType()),
    T.StructField("bike",        T.IntegerType()),
    T.StructField("tryke",       T.IntegerType()),
    T.StructField("others",      T.IntegerType()),
])

spark = (SparkSession.builder
         .appName("KafkaToCassandraCounts")
         .config("spark.cassandra.connection.host", "127.0.0.1")
         .getOrCreate())
spark.sparkContext.setLogLevel("WARN")

raw = (spark.readStream.format("kafka")
       .option("kafka.bootstrap.servers", "127.0.0.1:9092")
```

```
       .option("subscribe", "counts")
       .option("startingOffsets", "latest")
       .load())

parsed = (raw.selectExpr("CAST(value AS STRING) AS json_str")
          .select(F.from_json(F.col("json_str"), schema).alias("d"))
          .select("d.*")
          .filter(F.col("sensor_id").isNotNull() &
F.col("total").isNotNull()))


parsed = parsed.withColumn("raw_timeuuid_id", F.col("timeuuid_id"))

UUID36 =
"^[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[0-9a-fA-F]{4}-[0-9a-fA-F]{4}-[0-9a-fA-F]{1
2}$"
make_timeuuid = F.udf(lambda: str(uuid1()), T.StringType())

parsed = parsed.withColumn(
    "timeuuid_id",
    F.when(F.col("timeuuid_id").rlike(UUID36), F.col("timeuuid_id"))
     .otherwise(make_timeuuid())
)


console_q = (parsed.writeStream
             .format("console")
             .outputMode("append")
             .option("truncate", False)
             .option("numRows", 20)
             .start())

to_cass = parsed.select(
    "sensor_id",
    "timeuuid_id",
    "bike","bus","car","date_saved","jeepney",
    "lgu_code","others","time_saved","total","truck","tryke"
)

CHECKPOINT_DIR =
"/home/berna/spark-checkpoints/kafka_to_cassandra_counts_finalV"

def write_to_cassandra(df, batch_id: int):
```

```
    rows = df.count()
    (df.write.format("org.apache.spark.sql.cassandra")
       .mode("append")
       .options(keyspace="traffic_ks", table="traffic_counts")
       .save())
    print(f"[Spark] wrote micro-batch {batch_id} rows={rows}")

cass_q = (to_cass.writeStream
          .foreachBatch(write_to_cassandra)
          .outputMode("append")
          .trigger(processingTime="10 seconds")
          .option("checkpointLocation", CHECKPOINT_DIR)
          .start())
console_q.awaitTermination()
cass_q.awaitTermination()
```

Run Spark

Now that we created the needed python file to fetch the data from Kafka and send it to Cassandra using spark, we will now proceed in running spark inside the created directory for the fetch_stream_to_cassandra.py.

Inside the directory cd ~/kafka-spark-cass, run this command to start spark:

```
/opt/spark/bin/spark-submit  --packages org.apache.spark:spark-sql
-kafka-0-10_2.13:4.0.1,com.datastax.spark:spark-cassandra-connector_2.13:3.
5.1  --conf spark.cassandra.connection.host=127.0.0.1 /home/berna/kafka-
spark-cass/fetch_stream_to_cassandra.py
```

```
berna@DESKTOP-L0T6HMC: $ cd ~/kafka-spark-cass
berna@DESKTOP-L0T6HMC:~/kafka-spark-cass$ /opt/spark/bin/spark-submit --packages org.apache.spark:spark-sql-kafka-0-10_2.13:4.0.1,com.datastax.spark:spark-cas
sandra-connector_2.13:3.5.1 --conf spark.cassandra.connection.host=127.0.0.1 /home/berna/kafka-spark-cass/fetch_stream_to_cassandra.py
```

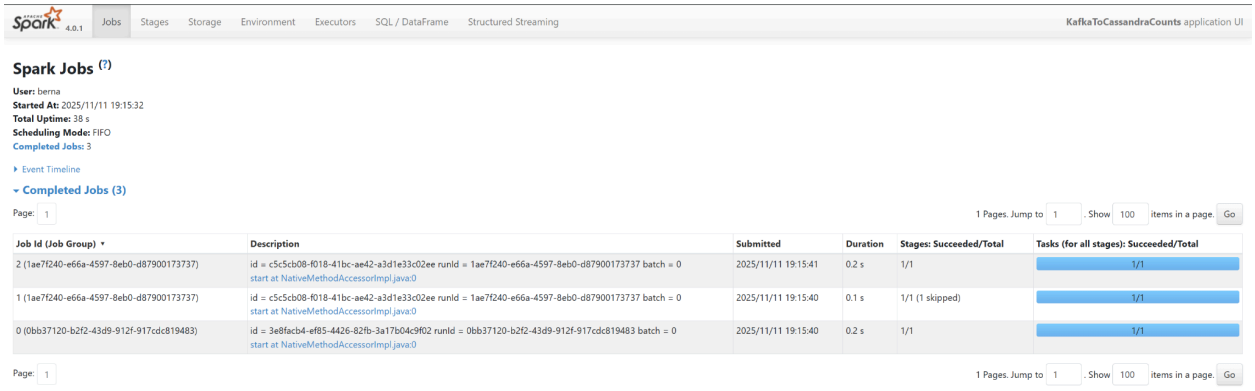
After running that line of code, the spark session will start and a batch with no data will appear, just like the image below:

```
25/11/11 19:15:34 INFO Executor: Adding file:/tmp/spark-dfd8951d-f954-430a-80ed-b1275ef7d3b1/userFiles-2734c410-3eb1-49ce-bfd8-6954c9438c5a/org.xerial.snappy_
snappy-java-1.1.10.7.jar to class loader default
25/11/11 19:15:34 INFO Utils: Successfully started service 'org.apache.spark.network.netty.NettyBlockTransferService' on port 34979.
25/11/11 19:15:34 INFO NettyBlockTransferService: Server created on 10.255.255.254:34979
25/11/11 19:15:34 INFO BlockManager: Using org.apache.spark.storage.RandomBlockReplicationPolicy for block replication policy
25/11/11 19:15:34 INFO BlockManagerMaster: Registering BlockManager BlockManagerId(driver, 10.255.255.254, 34979, None)
25/11/11 19:15:34 INFO BlockManagerMasterEndpoint: Registering block manager 10.255.255.254:34979 with 434.4 MiB RAM, BlockManagerId(driver, 10.255.255.254, 3
4979, None)
25/11/11 19:15:34 INFO BlockManagerMaster: Registered BlockManager BlockManagerId(driver, 10.255.255.254, 34979, None)
25/11/11 19:15:34 INFO BlockManager: Initialized BlockManager: BlockManagerId(driver, 10.255.255.254, 34979, None)
25/11/11 19:15:38 WARN ResolveWriteToStream: Temporary checkpoint location created which is deleted normally when the query didn't fail: /tmp/temporary-9c3002
71-6582-49f5-8fbf-6be60d1f0d64. If it's required to delete it under any circumstances, please set spark.sql.streaming.forceDeleteTempCheckpointLocation to tru
e. Important to know deleting temp checkpoint folder is best effort.
25/11/11 19:15:38 WARN ResolveWriteToStream: spark.sql.adaptive.enabled is not supported in streaming DataFrames/Datasets and will be disabled.
25/11/11 19:15:38 WARN ResolveWriteToStream: spark.sql.adaptive.enabled is not supported in streaming DataFrames/Datasets and will be disabled.
-------------------------------------------
Batch: 0
-------------------------------------------
+-----------+--------+---------+----------+----------+-----+---+---+-----+-------+----+-----+------+--------------+
|timeuuid_id|lgu_code|sensor_id|date_saved|time_saved|total|car|bus|truck|jeepney|bike|tryke|others|raw_timeuuid_id|
+-----------+--------+---------+----------+----------+-----+---+---+-----+-------+----+-----+------+--------------+
+-----------+--------+---------+----------+----------+-----+---+---+-----+-------+----+-----+------+--------------+

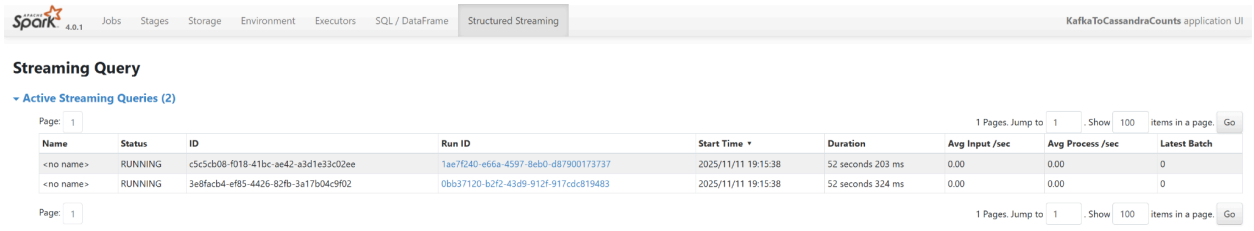
[Spark] wrote micro-batch 0 rows=0
```

Checking the Spark UI

We then proceed to check the Spark UI to see that there are no jobs yet, the first three completed jobs are the jobs created when we started spark using the line above.



Also going to the Structured Streaming tab, in the streaming query we see two, one for the console and the second one is for the Cassandra, and as you can see there are no jobs yet.



Check Cassandra Table

Now going to Cassandra, we see the tables by inputting, `SELECT * FROM traffic_counts;` that there is no saved data yet. This means we will now proceed in running the 10 producers so that the data using spark will be saved inside the Cassandra table.

```
cqlsh:traffic_ks>
cqlsh:traffic_ks>
cqlsh:traffic_ks> SELECT * FROM traffic_counts;

 sensor_id | timeuuid_id | bike | bus | car | date_saved | jeepney | lgu_code | others | time_saved | total | truck | tryke
-----------+-------------+------+-----+-----+------------+---------+----------+--------+------------+-------+-------+-------

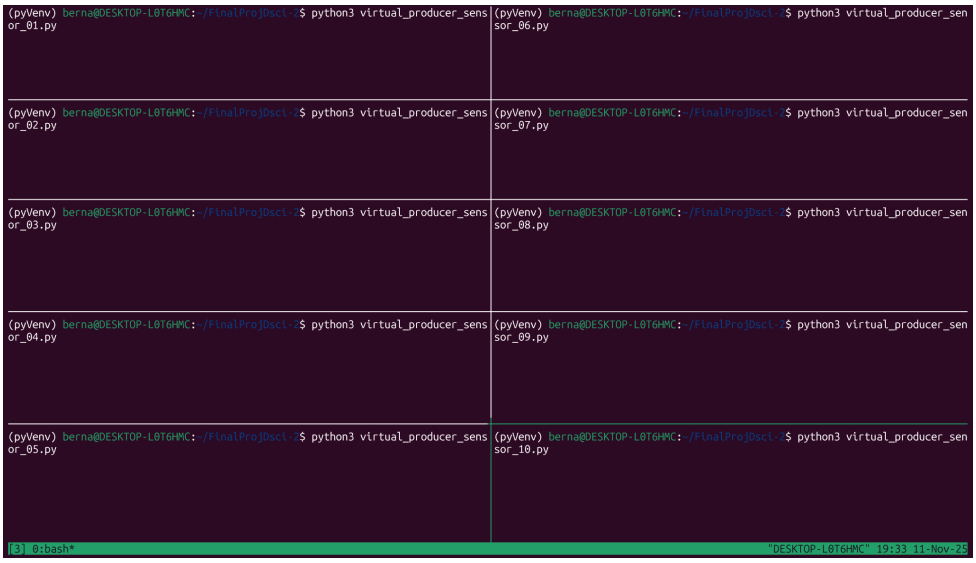
(0 rows)
cqlsh:traffic_ks> SELECT * FROM traffic_counts;

 sensor_id | timeuuid_id | bike | bus | car | date_saved | jeepney | lgu_code | others | time_saved | total | truck | tryke
-----------+-------------+------+-----+-----+------------+---------+----------+--------+------------+-------+-------+-------

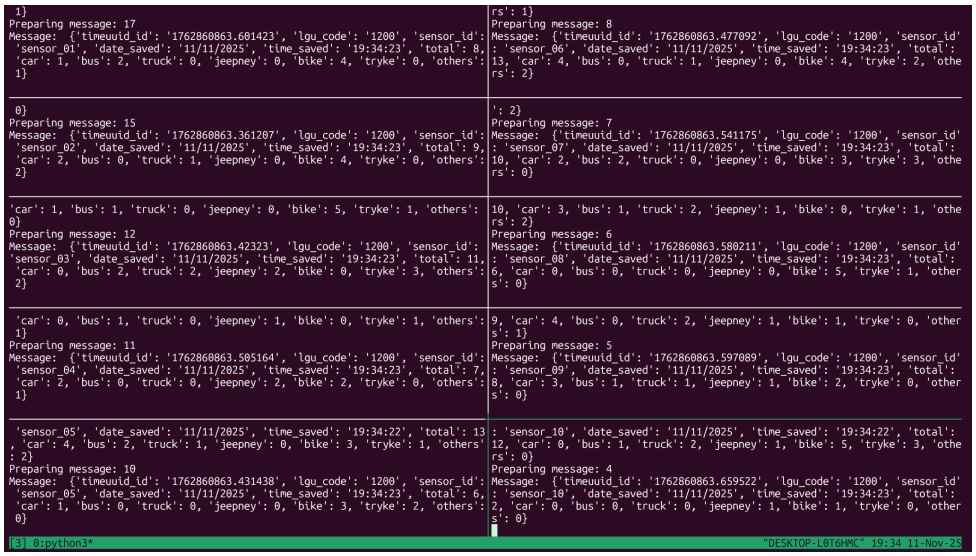
(0 rows)
cqlsh:traffic_ks>
cqlsh:traffic_ks>
```

Run 10 producers

In tmux, open 10 terminals and run the 10 producers again inside a python environment, or you can open 10 separate terminals.



As you can see the 10 producers are now simultaneously running inside tmux, but you can also run it on separate tabs with terminals, the usage of tmux is just for cleanliness and to easily show the 10 producers running.

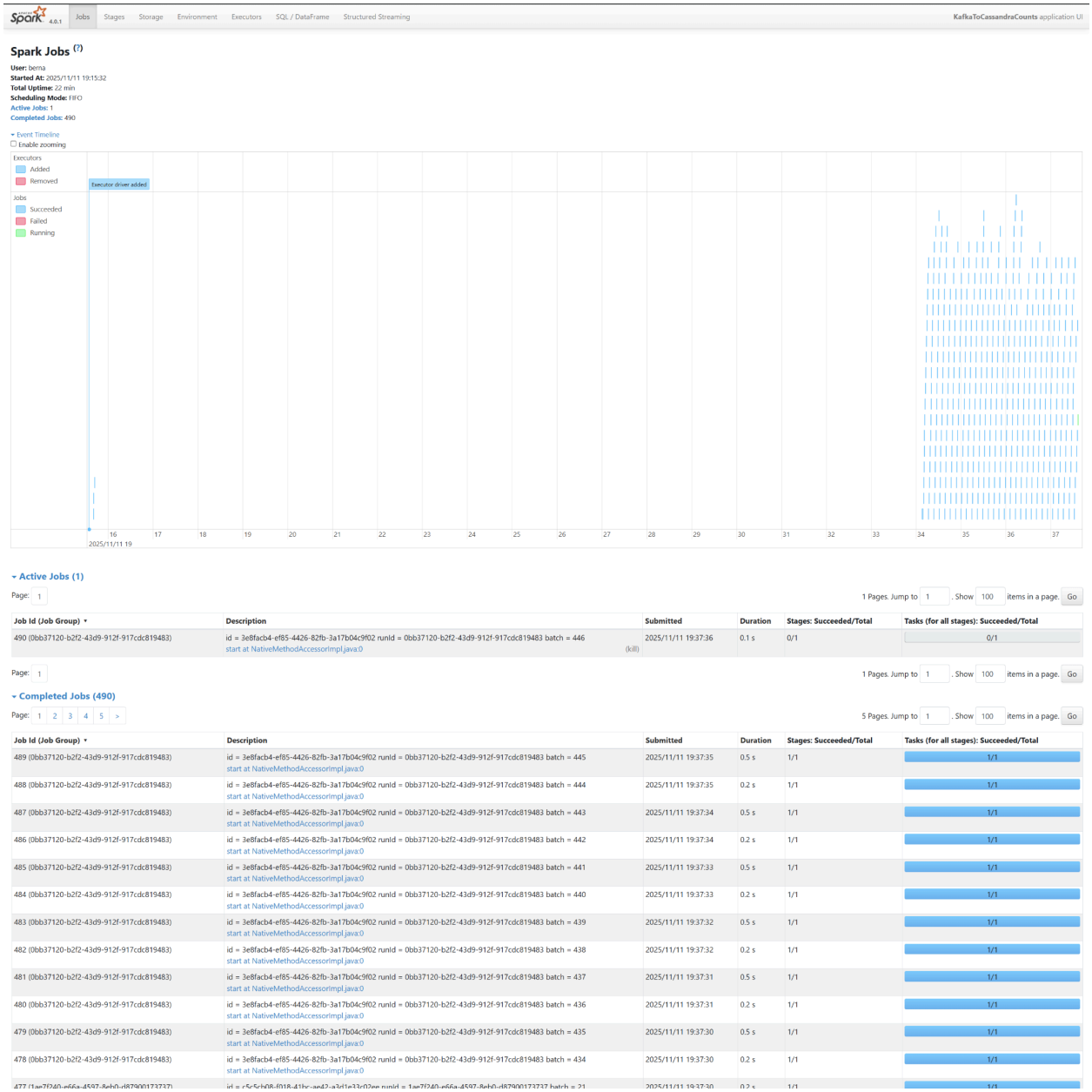


Also checking the consumer we can see that Kafka is now up and running.

```
berna@DESKTOP-L0T6HMC: $ cd FinalProjDsci-2
berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ source pyVenv/bin/activate
(pyVenv) berna@DESKTOP-L0T6HMC:~/FinalProjDsci-2$ python3 virtual_consumer.py
b'{"timeuuid_id": "1762860920.406543", "lgu_code": "1200", "sensor_id": "sensor_02", "date_saved": "11/11/2025", "time_saved": "19:35:20", "total": 7, "car":
0, "bus": 0, "truck": 1, "jeepney": 0, "bike": 2, "tryke": 3, "others": 1}'
b'{"timeuuid_id": "1762860920.458979", "lgu_code": "1200", "sensor_id": "sensor_03", "date_saved": "11/11/2025", "time_saved": "19:35:20", "total": 11, "car":
 3, "bus": 0, "truck": 0, "jeepney": 1, "bike": 4, "tryke": 1, "others": 2}'
b'{"timeuuid_id": "1762860920.466822", "lgu_code": "1200", "sensor_id": "sensor_05", "date_saved": "11/11/2025", "time_saved": "19:35:20", "total": 12, "car":
 1, "bus": 2, "truck": 2, "jeepney": 0, "bike": 5, "tryke": 0, "others": 2}'
b'{"timeuuid_id": "1762860920.511522", "lgu_code": "1200", "sensor_id": "sensor_06", "date_saved": "11/11/2025", "time_saved": "19:35:20", "total": 10, "car":
 1, "bus": 0, "truck": 0, "jeepney": 1, "bike": 4, "tryke": 3, "others": 1}'
b'{"timeuuid_id": "1762860920.536892", "lgu_code": "1200", "sensor_id": "sensor_04", "date_saved": "11/11/2025", "time_saved": "19:35:20", "total": 5, "car":
0, "bus": 0, "truck": 0, "jeepney": 0, "bike": 1, "tryke": 2, "others": 2}'
b'{"timeuuid_id": "1762860920.582489", "lgu_code": "1200", "sensor_id": "sensor_07", "date_saved": "11/11/2025", "time_saved": "19:35:20", "total": 14, "car":
 3, "bus": 2, "truck": 0, "jeepney": 1, "bike": 5, "tryke": 1, "others": 2}'
b'{"timeuuid_id": "1762860920.612315", "lgu_code": "1200", "sensor_id": "sensor_08", "date_saved": "11/11/2025", "time_saved": "19:35:20", "total": 17, "car":
 3, "bus": 2, "truck": 1, "jeepney": 2, "bike": 4, "tryke": 3, "others": 2}'
b'{"timeuuid_id": "1762860920.629317", "lgu_code": "1200", "sensor_id": "sensor_09", "date_saved": "11/11/2025", "time_saved": "19:35:20", "total": 10, "car":
 3, "bus": 0, "truck": 0, "jeepney": 1, "bike": 4, "tryke": 2, "others": 0}'
b'{"timeuuid_id": "1762860920.631123", "lgu_code": "1200", "sensor_id": "sensor_01", "date_saved": "11/11/2025", "time_saved": "19:35:20", "total": 12, "car":
 1, "bus": 1, "truck": 2, "jeepney": 1, "bike": 4, "tryke": 1, "others": 2}'
b'{"timeuuid_id": "1762860920.697109", "lgu_code": "1200", "sensor_id": "sensor_10", "date_saved": "11/11/2025", "time_saved": "19:35:20", "total": 10, "car":
 0, "bus": 1, "truck": 1, "jeepney": 0, "bike": 3, "tryke": 3, "others": 2}'
b'{"timeuuid_id": "1762860921.407351", "lgu_code": "1200", "sensor_id": "sensor_02", "date_saved": "11/11/2025", "time_saved": "19:35:21", "total": 14, "car":
```

Updated Spark UI and Console

Now checking the updated spark UI we can see that once we runned the 10 producers the Completed Jobs increased meaning that Spark is now streaming the data that is being produced by Kafka (Zaman, 2023).



Going back to Cassandra

Now going back to Cassandra, looking at the image we see taht in the first two inputs of checking for data inside the table, there is none. However once the producers were running, we can see that the python script is effectively streaming the data from Kafka and saving it inside the Cassandra Table.

Using the code `SELECT * FROM traffic_counts LIMIT 20;`, to limit to 20 entries, we see the data that was saved in Cassandra from Kafka using Spark:

```
cqlsh:traffic_ks> SELECT * FROM traffic_counts;

 sensor_id | timeuuid_id | bike | bus | car | date_saved | jeepney | lgu_code | others | time_saved | total | truck | tryke
-----------+-------------+------+-----+-----+------------+---------+----------+--------+------------+-------+-------+-------

(0 rows)
cqlsh:traffic_ks> SELECT * FROM traffic_counts;

 sensor_id | timeuuid_id | bike | bus | car | date_saved | jeepney | lgu_code | others | time_saved | total | truck | tryke
-----------+-------------+------+-----+-----+------------+---------+----------+--------+------------+-------+-------+-------

(0 rows)
cqlsh:traffic_ks>
cqlsh:traffic_ks> SELECT * FROM traffic_counts LIMIT 20;

 sensor_id | timeuuid_id                          | bike | bus | car | date_saved | jeepney | lgu_code | others | time_saved | total | truck | tryke
-----------+--------------------------------------+------+-----+-----+------------+---------+----------+--------+------------+-------+-------+-------
 sensor_01 | 1c3cc50a-bef3-11f0-8440-00155d16f12d |    0 |   0 |   4 | 11/11/2025 |       0 |     1200 |      0 |   19:39:39 |     4 |     0 |     0
 sensor_01 | 1c3cc2a7-bef3-11f0-affd-00155d16f12d |    0 |   0 |   1 | 11/11/2025 |       1 |     1200 |      1 |   19:39:38 |     5 |     0 |     2
 sensor_01 | 1c3cbfb7-bef3-11f0-a6c9-00155d16f12d |    5 |   0 |   0 | 11/11/2025 |       2 |     1200 |      2 |   19:39:37 |    10 |     1 |     0
 sensor_01 | 1c3cbd11-bef3-11f0-9f67-00155d16f12d |    1 |   2 |   0 | 11/11/2025 |       0 |     1200 |      2 |   19:39:36 |     9 |     1 |     3
 sensor_01 | 1c3cba74-bef3-11f0-a776-00155d16f12d |    1 |   2 |   0 | 11/11/2025 |       1 |     1200 |      1 |   19:39:35 |     9 |     2 |     2
 sensor_01 | 1c3cb7de-bef3-11f0-9c6e-00155d16f12d |    2 |   1 |   0 | 11/11/2025 |       2 |     1200 |      0 |   19:39:34 |     5 |     0 |     0
 sensor_01 | 1c3cb545-bef3-11f0-a5bd-00155d16f12d |    1 |   0 |   1 | 11/11/2025 |       0 |     1200 |      1 |   19:39:33 |     4 |     1 |     0
 sensor_01 | 1c3cb2a9-bef3-11f0-ad09-00155d16f12d |    2 |   1 |   3 | 11/11/2025 |       0 |     1200 |      0 |   19:39:32 |     7 |     1 |     0
 sensor_01 | 1c3cb017-bef3-11f0-985d-00155d16f12d |    0 |   0 |   4 | 11/11/2025 |       0 |     1200 |      1 |   19:39:31 |     7 |     2 |     0
 sensor_01 | 1c3cad8c-bef3-11f0-be07-00155d16f12d |    0 |   2 |   2 | 11/11/2025 |       0 |     1200 |      0 |   19:39:30 |     7 |     1 |     2
 sensor_01 | 164392fe-bef3-11f0-a124-00155d16f12d |    3 |   1 |   3 | 11/11/2025 |       1 |     1200 |      1 |   19:39:29 |    10 |     1 |     0
 sensor_01 | 1643918d-bef3-11f0-9b77-00155d16f12d |    4 |   0 |   4 | 11/11/2025 |       2 |     1200 |      0 |   19:39:28 |    14 |     1 |     3
 sensor_01 | 16439047-bef3-11f0-a074-00155d16f12d |    3 |   1 |   1 | 11/11/2025 |       2 |     1200 |      1 |   19:39:27 |    10 |     2 |     0
 sensor_01 | 16438efb-bef3-11f0-9d1d-00155d16f12d |    0 |   2 |   3 | 11/11/2025 |       2 |     1200 |      2 |   19:39:26 |    13 |     1 |     3
 sensor_01 | 16438d79-bef3-11f0-af89-00155d16f12d |    4 |   0 |   4 | 11/11/2025 |       2 |     1200 |      2 |   19:39:25 |    16 |     1 |     3
 sensor_01 | 16438be9-bef3-11f0-be75-00155d16f12d |    1 |   0 |   0 | 11/11/2025 |       1 |     1200 |      2 |   19:39:24 |     5 |     1 |     0
 sensor_01 | 16438aa7-bef3-11f0-8e0d-00155d16f12d |    1 |   1 |   0 | 11/11/2025 |       1 |     1200 |      2 |   19:39:23 |     7 |     1 |     1
 sensor_01 | 16438927-bef3-11f0-a887-00155d16f12d |    3 |   0 |   1 | 11/11/2025 |       1 |     1200 |      2 |   19:39:22 |     9 |     2 |     0
 sensor_01 | 16438786-bef3-11f0-9a4d-00155d16f12d |    5 |   0 |   4 | 11/11/2025 |       1 |     1200 |      1 |   19:39:21 |    14 |     0 |     3
 sensor_01 | 16438519-bef3-11f0-a850-00155d16f12d |    2 |   0 |   4 | 11/11/2025 |       0 |     1200 |      0 |   19:39:20 |     8 |     2 |     0

(20 rows)
cqlsh:traffic_ks>
```

Also by checking the total count through the command `SELECT COUNT(*) FROM traffic_counts;`, we see that so far there were a total of 3555 entries.

```
(20 rows)
cqlsh:traffic_ks> SELECT COUNT(*) FROM traffic_counts;

 count
-------
  3555

(1 rows)

Warnings :
Aggregation query used without partition key

cqlsh:traffic_ks>
```

Also to view all, we use the command `SELECT * FROM traffic_counts;`

```
cqlsh:traffic_ks> SELECT * FROM traffic_counts;

 sensor_id | timeuuid_id                          | bike | bus | car | date_saved | jeepney | lgu_code | others | time_saved | total | truck | tryke
-----------+--------------------------------------+------+-----+-----+------------+---------+----------+--------+------------+-------+-------+-------
 sensor_01 | 3a010630-bef3-11f0-8f29-00155d16f12d |    0 |   0 |   2 | 11/11/2025 |       1 |     1200 |      0 |   19:40:29 |     5 |     0 |     2
 sensor_01 | 3a010434-bef3-11f0-bd52-00155d16f12d |    3 |   2 |   2 | 11/11/2025 |       1 |     1200 |      0 |   19:40:28 |    12 |     1 |     3
 sensor_01 | 3a0101d1-bef3-11f0-85ce-00155d16f12d |    3 |   1 |   0 | 11/11/2025 |       1 |     1200 |      0 |   19:40:27 |     6 |     0 |     1
 sensor_01 | 3a010020-bef3-11f0-8f64-00155d16f12d |    2 |   1 |   3 | 11/11/2025 |       1 |     1200 |      0 |   19:40:26 |    10 |     1 |     2
 sensor_01 | 3a00fe70-bef3-11f0-b2e4-00155d16f12d |    0 |   1 |   4 | 11/11/2025 |       2 |     1200 |      0 |   19:40:25 |     7 |     0 |     0
 sensor_01 | 3a00fcbc-bef3-11f0-b852-00155d16f12d |    3 |   0 |   0 | 11/11/2025 |       2 |     1200 |      2 |   19:40:24 |    12 |     2 |     3
 sensor_01 | 3a00faff-bef3-11f0-a47e-00155d16f12d |    5 |   0 |   2 | 11/11/2025 |       0 |     1200 |      0 |   19:40:23 |    11 |     1 |     3
 sensor_01 | 3a00f96b-bef3-11f0-add1-00155d16f12d |    4 |   2 |   2 | 11/11/2025 |       1 |     1200 |      1 |   19:40:22 |    12 |     0 |     2
 sensor_01 | 3a00f78e-bef3-11f0-a8a1-00155d16f12d |    2 |   2 |   2 | 11/11/2025 |       1 |     1200 |      2 |   19:40:21 |    13 |     1 |     3
 sensor_01 | 3a00f5c3-bef3-11f0-88f8-00155d16f12d |    1 |   2 |   2 | 11/11/2025 |       0 |     1200 |      2 |   19:40:20 |     9 |     0 |     2
 sensor_01 | 340d09b4-bef3-11f0-a170-00155d16f12d |    5 |   1 |   1 | 11/11/2025 |       0 |     1200 |      2 |   19:40:19 |    12 |     1 |     2
 sensor_01 | 340d0899-bef3-11f0-b243-00155d16f12d |    4 |   1 |   0 | 11/11/2025 |       0 |     1200 |      1 |   19:40:18 |     7 |     1 |     0
 sensor_01 | 340d0763-bef3-11f0-828a-00155d16f12d |    1 |   1 |   0 | 11/11/2025 |       0 |     1200 |      1 |   19:40:17 |     6 |     0 |     3
 sensor_01 | 340d0647-bef3-11f0-ad60-00155d16f12d |    4 |   1 |   1 | 11/11/2025 |       2 |     1200 |      2 |   19:40:16 |    12 |     1 |     1
 sensor_01 | 340d0525-bef3-11f0-8564-00155d16f12d |    4 |   1 |   2 | 11/11/2025 |       1 |     1200 |      1 |   19:40:15 |    14 |     2 |     3
 sensor_01 | 340d040e-bef3-11f0-b7af-00155d16f12d |    3 |   0 |   3 | 11/11/2025 |       0 |     1200 |      0 |   19:40:14 |     8 |     2 |     0
 sensor_01 | 340d02f3-bef3-11f0-b8b1-00155d16f12d |    0 |   2 |   1 | 11/11/2025 |       2 |     1200 |      2 |   19:40:13 |    11 |     1 |     3
 sensor_01 | 340d01de-bef3-11f0-a461-00155d16f12d |    0 |   0 |   0 | 11/11/2025 |       0 |     1200 |      2 |   19:40:12 |     6 |     1 |     3
 sensor_01 | 340d00c6-bef3-11f0-9ec0-00155d16f12d |    2 |   2 |   1 | 11/11/2025 |       1 |     1200 |      2 |   19:40:11 |    10 |     2 |     0
```

```
sensor_09 | 754495e4-bef2-11f0-a907-00155d16f12d |    1 |   2 |   1 | 11/11/2025 |       2 |     1200 |       2 |   19:34:59 |    12 |     1 |     3
sensor_09 | 75449491-bef2-11f0-a5e2-00155d16f12d |    3 |   1 |   2 | 11/11/2025 |       1 |     1200 |       1 |   19:34:58 |    11 |     0 |     3
sensor_09 | 75449342-bef2-11f0-b636-00155d16f12d |    0 |   1 |   3 | 11/11/2025 |       1 |     1200 |       0 |   19:34:57 |     6 |     0 |     1
sensor_09 | 754491e5-bef2-11f0-8012-00155d16f12d |    4 |   0 |   0 | 11/11/2025 |       2 |     1200 |       1 |   19:34:56 |    10 |     1 |     2
sensor_09 | 75449087-bef2-11f0-ab55-00155d16f12d |    1 |   1 |   1 | 11/11/2025 |       0 |     1200 |       2 |   19:34:55 |     8 |     2 |     1
sensor_09 | 75448f2c-bef2-11f0-b86b-00155d16f12d |    2 |   0 |   4 | 11/11/2025 |       2 |     1200 |       2 |   19:34:54 |    11 |     1 |     0
sensor_09 | 75448dd5-bef2-11f0-973a-00155d16f12d |    3 |   2 |   2 | 11/11/2025 |       2 |     1200 |       1 |   19:34:53 |    14 |     1 |     3
sensor_09 | 75448c83-bef2-11f0-8136-00155d16f12d |    1 |   1 |   2 | 11/11/2025 |       2 |     1200 |       1 |   19:34:52 |    10 |     0 |     3
sensor_09 | 75448b2d-bef2-11f0-bc16-00155d16f12d |    2 |   2 |   1 | 11/11/2025 |       0 |     1200 |       1 |   19:34:51 |     8 |     1 |     1
sensor_09 | 754489d7-bef2-11f0-8e1f-00155d16f12d |    1 |   2 |   1 | 11/11/2025 |       2 |     1200 |       1 |   19:34:50 |    11 |     1 |     3
sensor_09 | 6f44bb50-bef2-11f0-b68a-00155d16f12d |    2 |   2 |   4 | 11/11/2025 |       2 |     1200 |       0 |   19:34:49 |    12 |     1 |     1
sensor_09 | 6f44ba04-bef2-11f0-8c6d-00155d16f12d |    4 |   0 |   3 | 11/11/2025 |       1 |     1200 |       2 |   19:34:48 |    12 |     1 |     1
sensor_09 | 6f44b8b2-bef2-11f0-a5ef-00155d16f12d |    0 |   0 |   0 | 11/11/2025 |       1 |     1200 |       2 |   19:34:47 |     6 |     0 |     3
sensor_09 | 6f44b769-bef2-11f0-b340-00155d16f12d |    1 |   0 |   3 | 11/11/2025 |       1 |     1200 |       2 |   19:34:46 |     9 |     0 |     2
sensor_09 | 6f44b5f0-bef2-11f0-a8b0-00155d16f12d |    3 |   1 |   0 | 11/11/2025 |       2 |     1200 |       2 |   19:34:45 |    11 |     1 |     2
sensor_09 | 6f44b3ce-bef2-11f0-bc1b-00155d16f12d |    5 |   2 |   0 | 11/11/2025 |       1 |     1200 |       1 |   19:34:44 |    11 |     1 |     1
sensor_09 | 6f44b28f-bef2-11f0-aa53-00155d16f12d |    4 |   2 |   0 | 11/11/2025 |       0 |     1200 |       1 |   19:34:43 |    11 |     2 |     2
sensor_09 | 6f44b14a-bef2-11f0-8c17-00155d16f12d |    1 |   0 |   2 | 11/11/2025 |       2 |     1200 |       1 |   19:34:42 |     9 |     2 |     1
sensor_09 | 6f44b000-bef2-11f0-b4a8-00155d16f12d |    2 |   0 |   4 | 11/11/2025 |       2 |     1200 |       1 |   19:34:41 |    11 |     1 |     1
sensor_09 | 6f44aeb2-bef2-11f0-82b3-00155d16f12d |    4 |   0 |   4 | 11/11/2025 |       2 |     1200 |       2 |   19:34:40 |    13 |     0 |     1
sensor_09 | 694c77c0-bef2-11f0-ba6d-00155d16f12d |    2 |   1 |   4 | 11/11/2025 |       2 |     1200 |       1 |   19:34:39 |    15 |     2 |     3
sensor_09 | 694c7660-bef2-11f0-9f00-00155d16f12d |    5 |   1 |   2 | 11/11/2025 |       0 |     1200 |       0 |   19:34:38 |    12 |     2 |     2
sensor_09 | 694c7516-bef2-11f0-b22c-00155d16f12d |    5 |   0 |   2 | 11/11/2025 |       0 |     1200 |       1 |   19:34:37 |     8 |     0 |     0
sensor_09 | 694c73cd-bef2-11f0-8b6e-00155d16f12d |    1 |   0 |   3 | 11/11/2025 |       1 |     1200 |       0 |   19:34:36 |     8 |     1 |     2
sensor_09 | 694c7281-bef2-11f0-a31c-00155d16f12d |    0 |   0 |   0 | 11/11/2025 |       0 |     1200 |       0 |   19:34:35 |     3 |     0 |     3
sensor_09 | 694c7140-bef2-11f0-a797-00155d16f12d |    0 |   0 |   0 | 11/11/2025 |       0 |     1200 |       2 |   19:34:34 |     6 |     1 |     3
sensor_09 | 694c6ffc-bef2-11f0-b4f0-00155d16f12d |    0 |   0 |   4 | 11/11/2025 |       0 |     1200 |       1 |   19:34:33 |     8 |     1 |     2
sensor_09 | 694c6eb0-bef2-11f0-a636-00155d16f12d |    5 |   0 |   2 | 11/11/2025 |       1 |     1200 |       0 |   19:34:32 |    10 |     1 |     1
sensor_09 | 694c6d5e-bef2-11f0-8ef1-00155d16f12d |    5 |   1 |   0 | 11/11/2025 |       1 |     1200 |       2 |   19:34:31 |    10 |     0 |     1
sensor_09 | 694c6c08-bef2-11f0-b065-00155d16f12d |    2 |   1 |   1 | 11/11/2025 |       1 |     1200 |       1 |   19:34:30 |     9 |     0 |     3
sensor_09 | 63543eae-bef2-11f0-afd2-00155d16f12d |    3 |   1 |   1 | 11/11/2025 |       1 |     1200 |       0 |   19:34:29 |     8 |     2 |     0
sensor_09 | 63543c2f-bef2-11f0-8a02-00155d16f12d |    2 |   0 |   4 | 11/11/2025 |       2 |     1200 |       2 |   19:34:28 |    12 |     2 |     0
sensor_09 | 63543a17-bef2-11f0-83ae-00155d16f12d |    3 |   2 |   2 | 11/11/2025 |       0 |     1200 |       0 |   19:34:27 |    11 |     2 |     2
sensor_09 | 6354379b-bef2-11f0-977a-00155d16f12d |    0 |   1 |   1 | 11/11/2025 |       0 |     1200 |       2 |   19:34:26 |     7 |     0 |     3
sensor_09 | 63543565-bef2-11f0-a4a3-00155d16f12d |    0 |   0 |   0 | 11/11/2025 |       0 |     1200 |       0 |   19:34:25 |     4 |     1 |     3
sensor_09 | 635432de-bef2-11f0-9740-00155d16f12d |    1 |   1 |   2 | 11/11/2025 |       0 |     1200 |       2 |   19:34:24 |     8 |     0 |     2
sensor_09 | 635430d9-bef2-11f0-ba0b-00155d16f12d |    2 |   1 |   3 | 11/11/2025 |       1 |     1200 |       0 |   19:34:23 |     8 |     1 |     0
sensor_09 | 63542ebc-bef2-11f0-bcba-00155d16f12d |    1 |   0 |   4 | 11/11/2025 |       1 |     1200 |       1 |   19:34:22 |     9 |     2 |     0
sensor_09 | 63542c06-bef2-11f0-acfc-00155d16f12d |    0 |   2 |   0 | 11/11/2025 |       2 |     1200 |       0 |   19:34:21 |     8 |     1 |     3
sensor_09 | 635428a1-bef2-11f0-848b-00155d16f12d |    2 |   0 |   2 | 11/11/2025 |       2 |     1200 |       2 |   19:34:20 |     8 |     0 |     0
sensor_09 | 5d612ad6-bef2-11f0-a87d-00155d16f12d |    0 |   2 |   1 | 11/11/2025 |       1 |     1200 |       1 |   19:34:19 |     5 |     0 |     0

(3755 rows)
cqlsh:traffic_ks>
```

Lastly, to see the current count after 3755 entries, we use again: `SELECT COUNT(*) FROM traffic_counts;`, and see that there is now 6754 entries.

```
(3755 rows)
cqlsh:traffic_ks> SELECT COUNT(*) FROM traffic_counts;

 count
-------
  6754

(1 rows)

Warnings :
Aggregation query used without partition key

cqlsh:traffic_ks>
```

Overall, this means that we have successfully incorporated Apache Kafka, Spark, and Cassandra in order to create this Big Data Streaming Project.

-END-

References:

Apache Cassandra. (n.d.-a). *Apache Cassandra | Apache Cassandra Documentation.* https://cassandra.apache.org/_/download.html

Apache Cassandra. (n.d.-b). *Support for Java 11 | Apache Cassandra Documentation.* https://cassandra.apache.org/doc/4.0/cassandra/new/java11.html

Apache Kafka. (2025a). *Apache Kafka — Quickstart.* https://kafka.apache.org/quickstart

Apache Kafka. (2025b, March 18). *Apache Kafka — Blog.* https://kafka.apache.org/blog

Apache Software Foundation. (2019). *Downloads | Apache Spark.* https://spark.apache.org/downloads.html

Ask Ubuntu. (2020). *Ask Ubuntu.* https://askubuntu.com/a/1266282

Confluent. (n.d.). *Apache Kafka and Java — Getting started tutorial.* https://developer.confluent.io/get-started/java/#prerequisites

DataCamp. (2024, March 13). *Apache Kafka for Beginners: A comprehensive guide.* https://www.datacamp.com/tutorial/apache-kafka-for-beginners-a-comprehensive-guide

DataMaking. (2019, October 20). Real-Time Spark Project | Data Analysis | Structured Streaming | Part 6.2 | DM | DataMaking. YouTube. https://www.youtube.com/watch?v=ZjsObNeUUd4&list=PLe1T0uBrDrfOYE8OwQvooPjmnP1zY3wFe&index=10

Dattell. (2025, March 26). *What is a Kafka Topic?* https://dattell.com/data-architecture-blog/what-is-a-kafka-topic/

GeeksforGeeks. (2017, October 4). *How to compress files in Linux | tar command.* https://www.geeksforgeeks.org/linux-unix/tar-command-linux-examples/

Gite, V. G. (2025, October 2). *How to install GPG (gnupg2) on a Debian Linux to fix gpg command not found error.* CyberCiti. https://www.cyberciti.biz/faq/installing-gnupg2-on-debian-linux-to-fix-bash-gpg-command-not-found-error/

Ludosanu, R. (2024, January 31). *What does sudo chown do?* Warp. https://www.warp.dev/terminus/sudo-chown

Smith, D. (2022, June 16). *Unix/Linux privilege management: Should you sudo? Here's what it does and why it's not enough.* BeyondTrust.

https://www.beyondtrust.com/blog/entry/unix-linux-privileged-management-should-you-sudo

TmuxCheatSheet. (n.d.). *Tmux cheat sheet & quick reference.* https://tmuxcheatsheet.com/

tutorialspoint. (n.d.). *Apache Kafka tutorial.* https://www.tutorialspoint.com/apache_kafka/index.htm

Verma, J. (2025, June 23). *What is .bashrc in Linux? Configuration explained.* DigitalOcean. https://www.digitalocean.com/community/tutorials/bashrc-file-in-linux

Zaman, A. U. (2023, May 5). Kafka with Spark Streaming —Different Approaches to Read Data. Medium; Plumbers Of Data Science. https://medium.com/plumbersofdatascience/kafka-with-spark-streaming-different-approaches-to-read-data-f38616c023b8